

JC4001: Distributed Systems

Topic 3: Processes

Slides Content Prepared by Dr Xiaonan Liu

xiaonan.liu@abdn.ac.uk

Delivery Team Member: Dr Shahzad Mumtaz

shahzad.mumtaz@abdn.ac.uk



Content

- Threads
- Virtualisation
- Clients
- Servers
- Code Migration

Threads

Threads: Introduction

Basic Idea

We build virtual processors in software, on top of physical processors:

- **Processor:** To execute a program, an operating system creates a number of virtual processors, and each one is used to run a different program. To keep track of these virtual processors, the operating system has a process table, containing entries to store CPU register values, memory maps, open files, accounting information, privileges, etc., these entries form a process context. Therefore, the processor provides a set of instructions along with the capability of automatically executing a series of those instructions.
- **Thread:** Like a process, a thread runs its own code independently of other threads. But unlike processes, threads don't aim for high concurrency. So, thread systems keep just the necessary information to let multiple threads share a CPU. A thread is like a minimal software processor that can run a series of instructions. Saving a thread context means stopping the current execution and keeping all the data needed to resume it later.
- **Process:** A process is basically a program being executed by the operating system's virtual processors. Multiple processes can share the same CPU simultaneously. The process context in software is like the processor context in hardware. A process is a software processor in which one or more threads can run. Running a thread means executing instructions within that thread's context.

Threads are smaller parts of processes that make distributed systems more efficient and easier to manage. They improve performance by focusing on what's necessary to keep things running smoothly without overloading the system.

Threads: Introduction

Context Switching: Contexts

- **Processor context:** It is the basic info the CPU needs to keep track of while running a series of instructions. This includes things like the stack pointer, addressing registers, and the program counter. It's the minimal set of values the processor needs to do its job and handle interruptions.
- **Thread context:** It is similar to the processor context but specific to threads. It includes the minimal set of values stored in registers and memory that are used for executing a thread's instructions. Essentially, it builds on the processor context and adds more details about the state of a particular thread.
- **Process context:** This is a bit more comprehensive. It includes everything in the thread context, but it also adds things like MMU (Memory Management Unit) register values. This context is used for executing a process, which might consist of multiple threads. So, it's like an all-encompassing record of what's going on with a process, making sure everything runs smoothly and independently.

Each of these contexts helps the operating system manage and switch between different tasks efficiently. The processor context is the most basic, the thread context adds more details specific to threads, and the process context is the most detailed, covering everything needed for a process to run.

Threads: Introduction

Context Switching: Observations

1. Threads share the same address space. This means that all threads within a process can access the same memory. Thread context switching can be done entirely independent of the operating system.
2. Process switching is generally (somewhat) more expensive as it involves getting the OS in the loop, i.e., when we switch between different processes, it takes more effort because the operating system has to get involved. This involves trapping to the kernel, which adds overhead.
3. Creating and destroying threads is much cheaper than doing so for processes. Starting or stopping a thread is less resource-intensive than doing the same with a full process. Threads are lighter-weight so that we can create and destroy them more quickly and easily.

Threads: Introduction

Why Use Multithreading: Some Simple Reasons

- **Avoid needless blocking:** In a single-threaded process, if a blocking system call happens, the whole process gets stuck. Imagine you're using a spreadsheet program, and you're constantly changing values. A spreadsheet needs to maintain relationships between cells, even across different spreadsheets. So, if you change one cell, all the dependent cells need to update automatically. This can start a bunch of calculations. If there's only one thread, it may be difficult to provide input while dependencies are being calculated. The easy solution is to have at least two threads of control: one for user interaction and one for updating the spreadsheet. Meanwhile, a third thread could be used to back up the spreadsheet to disk while the other two threads do their work.
- **Exploit parallelism:** It becomes possible to exploit parallelism when executing the program on a multiprocessor or multicore system. In that case, each thread is assigned to a different CPU or core, while shared data are stored in shared main memory. When properly designed, such parallelism can be transparent: the process will run equally well on a uniprocessor system, albeit slower.
- **Avoid process switching:** Structure large applications not as a collection of processes, but through multiple threads.

Threads: Introduction

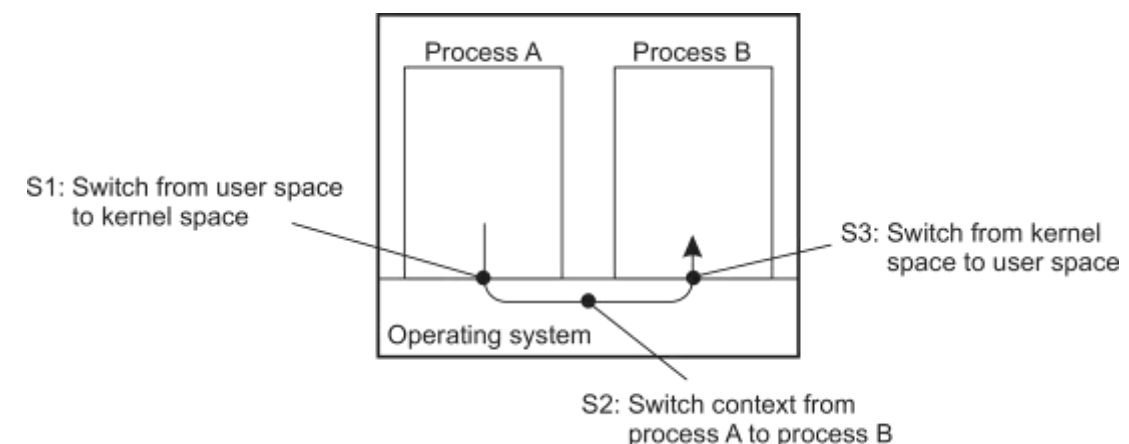
Avoid Process Switching: Avoid Expensive Context Switching

Multithreading is also useful in the context of large applications. Such applications are often developed as a collection of cooperating programs, each to be executed by a separate process. This approach is typical for a Unix environment. Cooperation between programs is implemented through interprocess communication (IPC) mechanisms. For Unix systems, these mechanisms typically include pipes, message queues, and shared memory segments. The major drawback of all interprocess communication mechanisms is that communication often requires relatively extensive context switching.

S1: Interprocess communication requires kernel to coordinate, a process will generally first have to switch from user mode to kernel mode. This involves changing the memory map in the Memory Management Unit (MMU) and clearing the Translation Lookaside Buffer (TLB).

S2: Within the kernel, a process context switch takes place.

S3: The other party can be activated by switching from kernel mode to user mode again, which again requires changing the MMU map and clearing the TLB.



You can build an application where different parts run as separate threads. These threads communicate using shared data. Sometimes, thread switching can be done entirely in user space, but in other cases, the kernel knows about the threads and handles scheduling. This can lead to a performance improvement.

Threads: Introduction

Avoid Process Switching: Avoid Expensive Context Switching

Trade-offs Using Multithreading

- When threads run, they share the same memory space. This means if one thread messes up and writes some bad data, it can affect all the other threads.
- The operating system and hardware don't step in to protect the memory used by threads from being messed with by other threads.
- Switching between threads can be faster than switching between processes.

Summary

- Although using threads can be faster and more efficient, it also comes with some risks, mainly because they share the same space and the system doesn't protect them from each other.

Threads: Introduction

The Cost of a Context Switch: How to measure?

- One of the approach is to use the clock ticks, since handling clock ticks has become more or less ubiquitous in operating systems
- A clock handler gets activated every T milliseconds by a clock interrupt. T is usually between 0.5 and 20 milliseconds, which means the interrupts happen 2000 to 50 times per second. The clock handler helps with various tasks like timing, CPU usage, sending alarm signals, and making sure tasks share the CPU fairly. By simply varying the frequency of these interrupts happen, we can see the overhead they cause.

Threads: Introduction

The Cost of a Context Switch: Consider a simple clock-interrupt handler

- **Direct costs:** It is the time taken to do the context switch, run the handler, and switch back to the task.
- **Indirect costs:** It includes everything else, mainly cache disruptions. For Intel processors, context switching takes about 0.5–1 microsecond, and the handler takes about 0.5–7 microseconds, depending on how it's done.

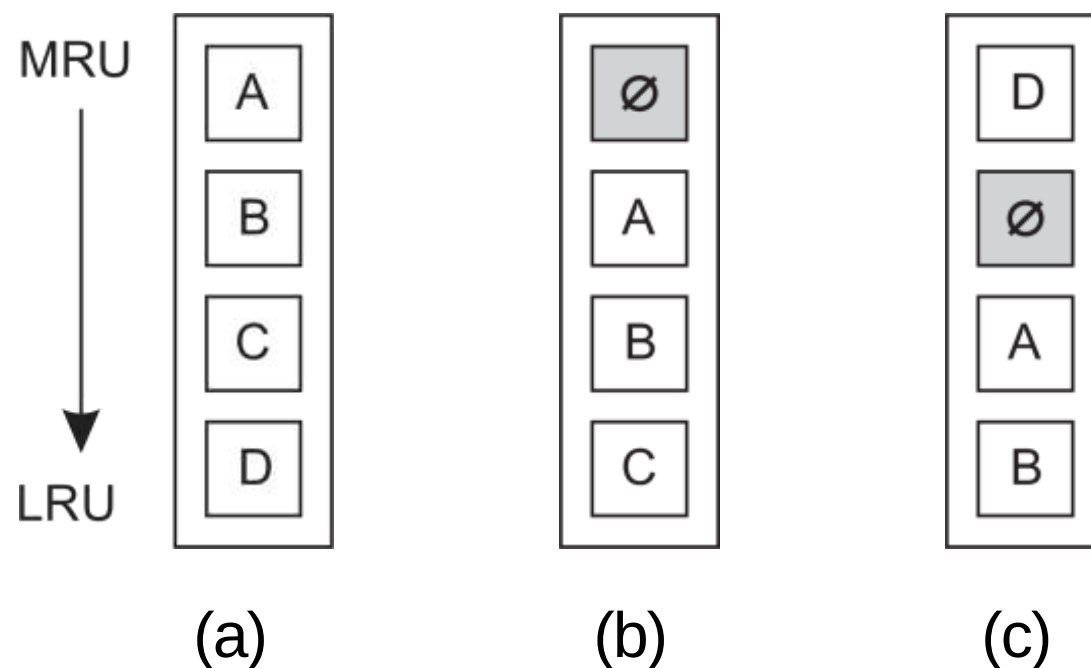
Direct overhead isn't the big issue. It shows that context switching can really mess up the cache, leading to performance drops compared to before the interrupt, referring to the indirect cost.

Threads: Introduction

The Cost of a Context Switch: Consider a simple clock-interrupt handler

How data is organized in the cache

- The cache kicks out the least recently used data block when it needs space
- Even a simple interrupt can cause a big and lasting reorganisation of the cache, which hurts the application's performance.



(a) The cache before an interrupt.

(b) After the interrupt, block D is removed, leaving a hole.

(c) When you access block D again, it gets copied back into the cache, possibly kicking out block C, and so on.

Threads: Introduction

A Simple Example in Python: Example 1

The example demonstrates how to separate processes in Python using the multiprocessing package. The main part of the example is the sleeping function, which makes the calling process sleep for a randomly chosen number of seconds.

To create two processes, we use the Process operation on lines 14 and 15 and then start each of them. The join operation tells the main process to wait until the new processes are done.

```
1  ✓ import multiprocessing as mp
2  import time
3  import random
4
5  ✓ def sleeper(name: str):
6      t = time.gmtime()
7      s = random.randint(1, 20)
8      print(f"{t.tm_min}:{t.tm_sec} {name} is going to sleep for {s} seconds")
9      time.sleep(s)
10     t = time.gmtime()
11     print(f"{t.tm_min}:{t.tm_sec} {name} has woken up")
12
13  ✓ if __name__ == "__main__":
14      p = mp.Process(target=sleeper, args=("eve",))
15      q = mp.Process(target=sleeper, args=("bob",))
16      p.start(); q.start()
17      p.join(); q.join()
18
```

Output

```
46:34 eve is going to sleep for 11 seconds
46:34 bob is going to sleep for 18 seconds
46:45 eve has woken up
46:52 bob has woken up
```

Threads: Introduction

A Simple Example in Python: Example 2

We start two processes, named eve and bob. Each process then starts three threads, each running the sleeping function. The big difference now is that we have a shared variable, `shared_x`.

```
1  from __future__ import annotations
2  import multiprocessing as mp
3  import threading
4  import time
5  import random
6  import os
7  THREADS_PER_PROCESS: int = 3
8  def sleeping(name: str, shared: dict[str, int], lock: threading.Lock) -> None:
9      t = time.gmtime()
10     s = random.randint(1, 5)
11     print(f"{t.tm_min:02d}:{t.tm_sec:02d} [{os.getpid()}] {name} is going to sleep for {s} seconds")
12     time.sleep(s)
13     with lock:
14         shared["x"] += 1
15         seen = shared["x"]
16     t = time.gmtime()
17     print(f"{t.tm_min:02d}:{t.tm_sec:02d} [{os.getpid()}] {name} has woken up, shared_x now {seen}")
18
19 def sleeper(proc_name: str, result_queue: mp.Queue) -> None:
20     shared = {"x": random.randint(10, 99)}
21     lock = threading.Lock()
22     print(f"[{os.getpid()}] {proc_name} starts with shared_x={shared['x']}")
23     threads: list[threading.Thread] = []
24     for i in range(THREADS_PER_PROCESS):
25         th = threading.Thread(
26             target=sleeping,
27             args=(f"{proc_name}/t{i}", shared, lock),
28             daemon=True,
29         )
30         threads.append(th)
31         th.start()
32     for th in threads:
33         th.join()
34     print(f"[{os.getpid()}] {proc_name} final shared_x={shared['x']}")
35     result_queue.put((proc_name, shared["x"]))
36
37 if __name__ == "__main__":
38     mp.set_start_method("spawn", force=True)
39     results: mp.Queue = mp.Queue()
40     p1 = mp.Process(target=sleeper, args=("eve", results))
41     p2 = mp.Process(target=sleeper, args=("bob", results))
42     p1.start(); p2.start()
43     p1.join(); p2.join()
44     proc_results = dict([results.get(), results.get()])
45     print("[Parent] Per-process shared_x values:", proc_results)
46
```

3 threads

Threads: Introduction

A Simple Example in Python: Example 2

The output here shows that `shared_x` is shared among the threads in one process but not between the two processes, eve and bob. So, each process has its own instance of `shared_x`.

Output

```
[20700] eve starts with shared_x=92
31:55 [20700] eve/t0 is going to sleep for 1 seconds
31:55 [20700] eve/t1 is going to sleep for 4 seconds
31:55 [20700] eve/t2 is going to sleep for 5 seconds
[9424] bob starts with shared_x=32
31:55 [9424] bob/t0 is going to sleep for 1 seconds
31:55 [9424] bob/t1 is going to sleep for 5 seconds
31:55 [9424] bob/t2 is going to sleep for 1 seconds
31:56 [20700] eve/t0 has woken up, shared_x now 93
31:56 [9424] bob/t0 has woken up, shared_x now 33
31:56 [9424] bob/t2 has woken up, shared_x now 34
31:59 [20700] eve/t1 has woken up, shared_x now 94
32:00 [20700] eve/t2 has woken up, shared_x now 95
[20700] eve final shared_x=95
32:00 [9424] bob/t1 has woken up, shared_x now 35
[9424] bob final shared_x=35
[Parent] Per-process shared_x values: {'eve': 95, 'bob': 35}
```

The output also shows that the sleep function works at both the thread and process levels. Each thread sleeps for a few seconds, but the process hosting the thread isn't blocked when sleep is called. Instead, another thread runs (and then also calls sleep). How this works is transparent to the programmer. Also, it's transparent whether different threads run on different CPU cores, if available. If we started 1000 threads per process, we'd still see accurate timing.

Threads: Introduction

Threads and Operating Systems

Threads usually come in what's called a thread package. This package includes operations to create and destroy threads and manage synchronisation tools. There are two main ways to implement a thread package:

- **User-level Thread Library** – which runs entirely in user space.
- **Kernel-level** – which handles threads and their scheduling.

Threads: Introduction

Threads and Operating Systems: User-Level Thread Library

Advantages

- Creating and destroying threads is cheap. Since all thread management occurs in the user's address space, the main cost of creating a thread is the memory required for its stack. Similarly, destroying a thread is mostly about freeing up that memory. Both of these operations are pretty inexpensive.
- Switching between threads can be done really quickly, often in just a few instructions. You only need to save the CPU register values and then reload them for the new thread. There's no need to change memory maps or do CPU accounting. Thread context switching usually occurs when threads need to synchronise, such as when accessing shared data. But, as mentioned earlier, much of the overhead from context switching comes from messing up memory caches.

Disadvantages

- User-level threads have a big drawback with the many-to-one threading model, where multiple threads are mapped to a single schedulable entity. If a thread makes a blocking system call, the entire process, including all its threads, is blocked. Threads are great for structuring large applications into parts that can run at the same time. But if one part blocks on I/O, it shouldn't stop the rest.
- Threads are often used to handle multiple external events at once. They block and wait for specific events (like I/O operations). But, if the kernel can't see or manage these threads individually, it can't properly signal events to them. In such cases, user-level threads aren't very helpful.

Threads: Introduction

Threads and Operating Systems: Kernel-Level Solution

The whole idea is to have the kernel contain the implementation of a thread package. This means that all operations return as system calls:

- Operations that block a thread are no longer a problem: the kernel schedules another available thread within the same process.
- Handling external events is simple: the kernel (which catches all events) schedules the thread associated with the event.
- The problem is (or used to be) the loss of efficiency because each thread operation requires a trap to the kernel.

Threads: Introduction

Threads and Operating Systems: Conclusion – but

Try to mix user-level and kernel-level threads into a single concept, however, performance gain has not turned out to generally outweigh the increased complexity.

Threads: Introduction

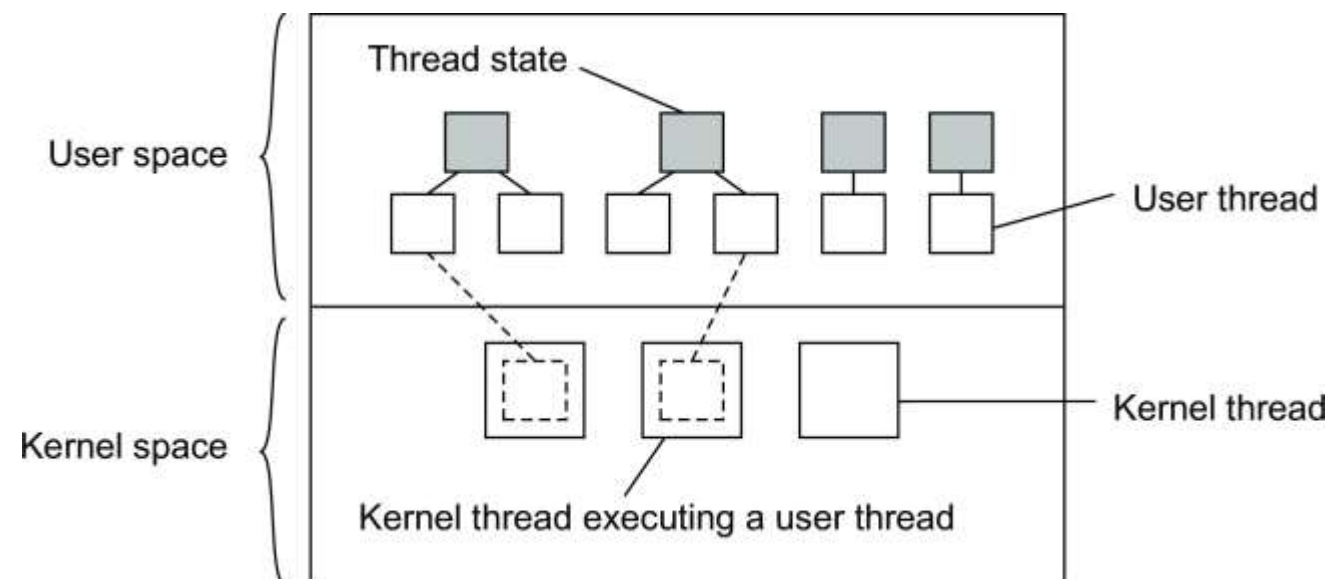
Threads and Operating Systems: User-level and Kernel-level Threads Combined

Basic Idea

Introduce a two-level threading approach: kernel threads that can execute user-level threads.

A kernel thread runs in the context of a single process, and there can be several kernel threads per process. In addition to managing kernel threads, a runtime system also offers a user-level thread package. This package provides all the usual operations for creating and destroying threads and includes tools for thread synchronisation. The important issue is that the thread package is implemented entirely in user space: all operations on threads are carried out without the kernel's help.

This thread package can be shared by multiple kernel threads, which means each kernel thread can run its own user thread. So, when building multithreaded applications, you create both user and kernel threads and then assign each user thread to a kernel thread. This setup combines the best of both worlds, giving us flexibility and efficiency.



Threads: Introduction

Threads and Operating Systems: User-level and Kernal-level Threads Combined

Principle Operation (1/2)

- When a user thread makes a system call, the kernel thread running that user thread gets blocked. But the user thread stays attached to that kernel thread. We can think it like the user thread is stuck waiting in line, and the kernel thread is the person holding its place.
- The kernel can then pick another kernel thread that has a runnable user thread attached to it and start running that one. Note that, this user thread can switch to any other runnable user thread that's in user space. So, it's like switching lines at the supermarket if another line is moving faster.
- If a user thread calls a blocking operation, it will do a context switch to another runnable user thread but still stay attached to the same kernel thread. It's like if the person holding your place in line switches to hold someone else's place for a bit but comes back to you later.
- If there are no user threads left to run, a kernel thread might just sit idle or even get removed by the kernel. It's like if everyone has left the store, the person holding the place in line can go home.

Threads: Introduction

Threads and Operating Systems: User-level and Kernal-level Threads Combined

Principle Operation (2/2)

- It helps ensure that if one part of a process gets blocked, it doesn't stop everything else from running. It makes the system more efficient and flexible, especially in handling multiple tasks simultaneously.
- It's important to understand that using threads is just one way to manage simultaneous and concurrent executions within an application. In practice, many applications are built as a bunch of concurrent processes that work together using the interprocess communication tools provided by the operating system.
- Using processes instead of threads has a big advantage: it separates the data space. Each process works on its own data and is protected from interference by other processes through the operating system. This separation is a big deal because programming with threads can be really tricky. When you use threads, you have to manage concurrent access to shared data all by yourself. But with processes, the hardware helps protect data spaces. If a process tries to access data outside its allocated memory, the hardware raises an exception, and the operating system handles it. This kind of protection isn't available for threads working within the same process.

Threads: In Distributed Systems

Using Threads at the Client Side

Multithreaded web client

Hiding network latencies:

- Web browser scans an incoming HTML page, and finds that more files need to be fetched.
- Each file is fetched by a separate thread, each doing a (blocking) HTTP request.
- As files come in, the browser displays them.

Multiple request-response calls to other machines (RPC)

- A client does several calls at the same time, each one by a different thread.
- It then waits until all results have been returned.
- Note: if calls are to different servers, we may have a linear speed-up.

Threads: In Distributed Systems

Multithreaded clients: does it help?

Thread-level parallelism (TLP):

Let c_i denote the fraction of time that exactly i threads are being executed simultaneously.

$$TLP = \frac{\sum_{i=1}^N i \cdot c_i}{1 - c_0}$$

with N the maximum number of threads that (can) execute at the same time.

Practical Measurements

A typical Web browser has a TLP value between 1.5 and 2.5 $\Rightarrow$ This means that to fully take advantage of parallelism, the client machine should have two or three cores or processors. Threads are primarily used for logically organizing browsers.

Threads: In Distributed Systems

Using Threads at the Server Side

Improve Performance

- Starting a thread is cheaper than starting a new process. This means it's much faster and uses fewer resources, allowing us to handle more tasks without slowing down the system.
- If you have a single-threaded server, it doesn't scale well with multiprocessor systems.
- As with clients – hide network latency: Just like with clients, servers can hide network delays by moving on to the next request while still processing the previous one. This keeps things running smoothly and efficiently, so users aren't left waiting.

Better Structure

- Most servers have high I/O demands. Using simple, well-understood blocking calls simplifies the structure.
- Multithreaded programs tend to be smaller and easier to understand due to simplified flow of control.

Threads: In Distributed Systems

Why Multithreading is Popular: Organisation

Dispatch/worker model: File server

A file server has to wait for the disk, which can block the process.

Normally, the server waits for a request, processes it, and then sends back a response.

Threads: In Distributed Systems

Why Multithreading is Popular: Organisation

Dispatch/worker model: File server

A file server has to wait for the disk, which can block the process. Normally, the server waits for a request, processes it, and then sends back a response

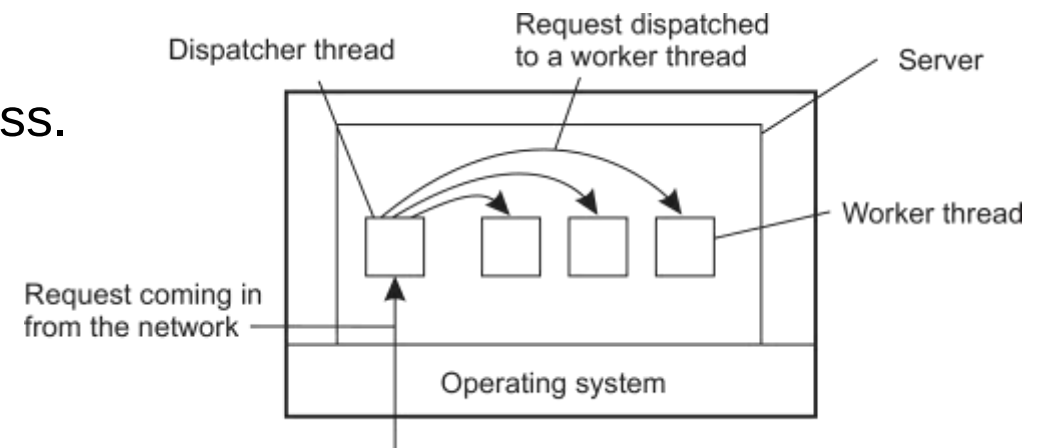
How it works:

One thread, called the dispatcher, reads incoming requests. Clients send these requests to a well-known endpoint. The dispatcher checks out the request and then hands it off to an idle worker thread to handle it.

Not only streamline the server's workflow but also makes sure the server can keep working on new requests while other threads are waiting for the disk or other resources. This is especially useful with multicore processors, where threads can run truly in parallel, making the server much more efficient.

When a worker thread in a file server performs a blocking read on the local file system, it might get suspended while waiting for data from the disk. If that happens, another thread takes over. It could be the dispatcher gets more work, or another worker thread that's ready to run.

If the file server was written without threads. It could operate as a single thread. The server would get a request, process it completely, and then move on to the next one. While waiting for the disk, the server does nothing else. This means it can't handle requests from other clients during that time. If the server runs on a dedicated machine, the CPU just sits idle while the server waits for the disk. The result is that the server processes fewer requests over time. Threads, on the other hand, enhance performance since each thread operates sequentially in the usual way.



Threads: In Distributed Systems

Why Multithreading is Popular: Organisation

A single-threaded server running as a big finite-state machine

When a request comes in, the single thread processes it. If the request can be satisfied from the in-memory cache, great. If not, the thread schedules a nonblocking (asynchronous) disk operation and moves on to check other requests. When the disk operation is done, the operating system notifies the thread, which then resumes processing the original request and eventually sends a response using another nonblocking call.

In this design, the simplicity of the sequential process is lost. Every time the thread needs to do a blocking operation, it has to record its progress and any additional state information. Then, it starts the operation and moves on to other tasks, like new requests or post-processing completed operations. If there's no work left, the thread might block. This setup mimics multiple threads and their stacks but in a more complex way. The server runs like a finite-state machine, reacting to events based on its current state.

The key point here is that threads let us keep the simplicity of sequential processes with blocking system calls while achieving parallelism. Blocking calls make programming easier since they look like normal procedure calls. Multiple threads enable parallelism, enhancing performance. A single-threaded server is simpler but can significantly limit the number of requests handled per time unit. The finite-state machine approach achieves high performance with parallelism but uses nonblocking calls, which are harder to program and maintain. The three models are summarized below.

Instead of threads, we could use multiple processes to organize a server. This setup, a multiprocess server, offers better protection against accidental data access. However, if the processes need to communicate a lot, it could impair performance compared to using threads.

Model	Characteristics
Multithreading	Parallelism, blocking system calls
Single-threaded process	No parallelism, blocking system calls
Finite-state machine	Parallelism, nonblocking system calls

Virtualisation

Virtualisation

Observation

- Threads and processes are basically ways to do multiple things at once. They let us create programs that look like they're running simultaneously. On a single-processor (or single-core) computer, this simultaneous execution is actually just an illusion. Since there's only one CPU, it can only run one instruction from one thread or process at a time. But by rapidly switching between threads and processes, it creates the parallelism.
- This idea of using one CPU to pretend there are more extends to other resources too, which we call resource virtualisation.
- Virtualisation has been around for many decades but has become especially important as distributed computer systems have become more common and complex.

Virtualisation

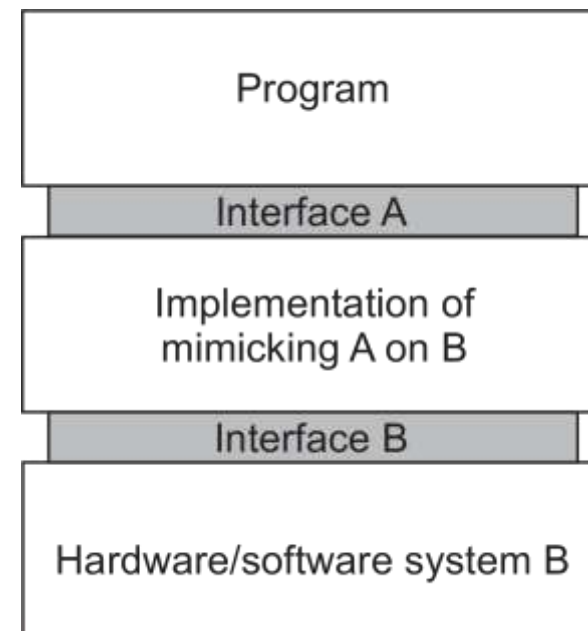
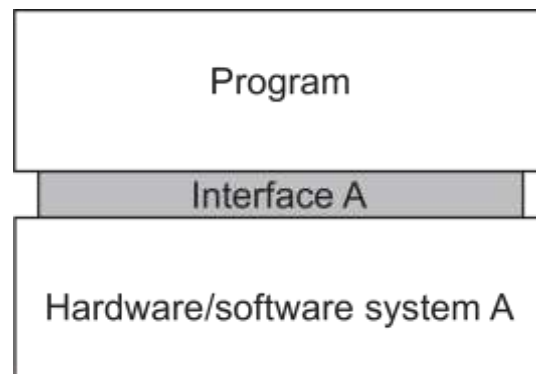
Virtualisation is Important

- While hardware and low-level software evolved quickly, higher-level software like middleware and applications tended to be more stable. This created a situation where old software couldn't keep up with the platforms it depended on. Virtualisation helps here by allowing old interfaces to run on new platforms, making these new platforms immediately compatible with a lot of existing software.
- Networking has become everywhere. It's hard to imagine a modern computer not being connected to a network. This connectivity means system administrators have to manage a large and varied collection of servers, each running different applications accessed by clients. Virtualisation simplifies this by letting each application run on its own virtual machine, possibly with its own libraries and operating system, all on a common platform.
- This type of virtualisation offers great portability and flexibility. For example, to manage content delivery networks that support dynamic content replication, it's much easier if edge servers support virtualisation. This allows a whole site, including its environment, to be dynamically copied. This approach is still valid today, and portability is one of the biggest reasons why virtualisation is so important in many distributed systems.
- Virtualisation provides an extra layer of isolation for code, which is especially relevant for cloud computing. This isolation makes sure that a failure or security attack doesn't affect the whole system.

Virtualisation

Principle: mimicking interfaces

- In practice, every distributed computer system provides a programming interface for higher-level software.
- These interfaces can be anything from the basic instruction set offered by a CPU to the huge collection of application programming interfaces (APIs).
- At its core, virtualisation is about extending or replacing an existing interface to make it behave like another system

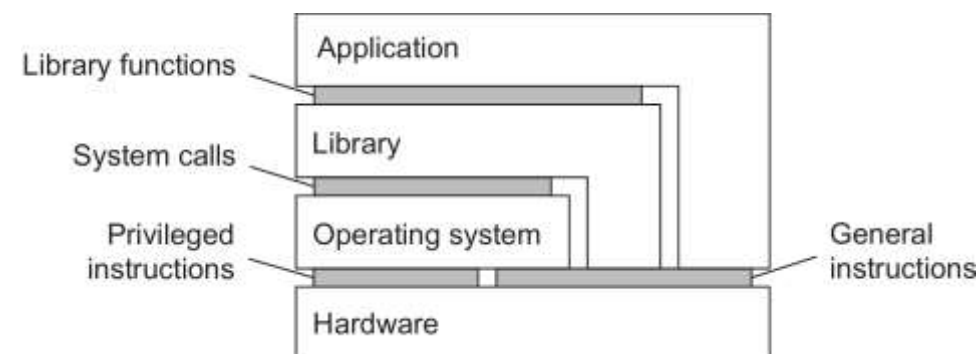


Virtualisation

Mimicking Interfaces

Four types of interfaces at three different levels

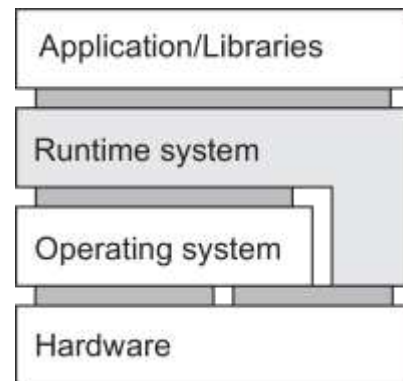
1. Instruction set architecture: This is the interface between the hardware and software, basically the set of machine instructions, with two subsets:
 - Privileged instructions: allowed to be executed only by the operating system.
 - General instructions: can be executed by any program.
2. System calls as offered by an operating system for the software to interact with hardware and system resources.
3. Library calls, known as an application programming interface (API), system calls are hidden behind an API to make things simpler for programmers.



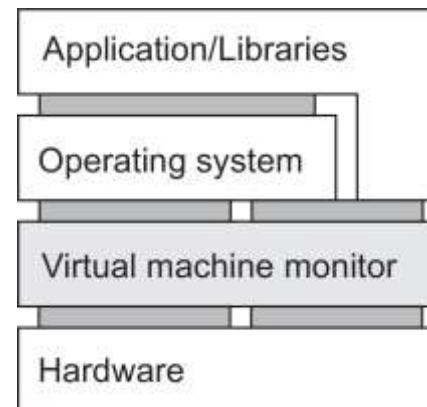
The core idea of virtualisation is to mimic the behaviour of these interfaces. This way, software thinks it's running on one type of system when it's actually running on a different one.

Virtualisation

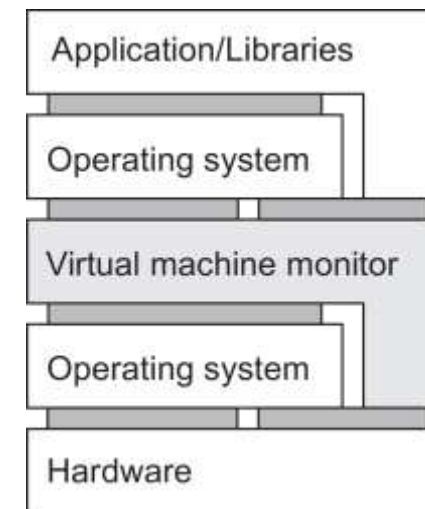
Ways of Virtualisation



a. Process VM



b. Native VMM



c. Hosted VMM

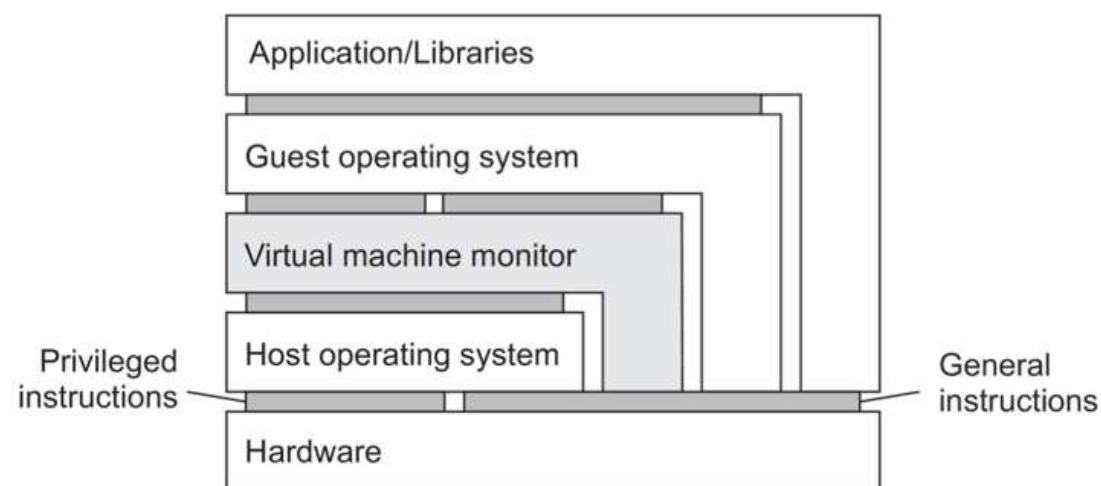
- a. We can create a runtime system that provides an abstract set of instructions for running applications. These instructions can be interpreted, like running Windows applications on Unix systems. In this case, the emulator also has to mimic system calls. This type of virtualisation is called a process virtual machine because it's designed for a single process. A separate set of instructions, an interpreter/emulator, running atop an OS.
- b. Build a system that acts as a layer between the original hardware and the software, offering the complete instruction set of the hardware. This is known as a native virtual machine monitor because it's implemented directly on the hardware. This setup allows multiple and different guest operating systems to run independently and concurrently on the same machine.
- c. A native virtual machine monitor also needs to manage access to various resources like storage and networks, which means it has to implement device drivers, much like an operating system. Instead of reinventing a method, a hosted virtual machine monitor runs on top of a trusted host operating system. This way, it can use the existing facilities provided by the host OS. Usually, the virtual machine monitor needs special privileges instead of running as a regular application. This hosted approach is popular in modern distributed systems like data centers and cloud services.

It shows that virtual machines are crucial for reliability and security in distributed systems. They allow isolating an entire application and its environment, so a failure or security attack doesn't affect the whole machine. Plus, virtual machines improve portability by decoupling hardware and software, making it easier to move environments from one machine to another.

Virtualisation

Zooming into VMs: Performance

Refining the Organisation



- Privileged Instruction: if and only if executed in user mode, it causes a trap to the operating system.
- Nonprivileged instruction: the rest

Special Instructions

- Control-sensitive instruction: may affect configuration of a machine (e.g., one affecting relocation register or interrupt table).
- Behaviour-sensitive instruction: effect is partially determined by context (e.g., POPF set an interrupt-enabled flag, but only in system mode).

Virtualisation

Condition for Virtualisation

Necessary Condition

For any conventional computer, a virtual machine monitor may be constructed if the set of sensitive instructions for that computer is a subset of the set of privileged instructions. This means that if we can catch sensitive instructions when they're running in user mode, we can safely run all the other instructions directly on the hardware. When designing instruction sets, if we ensure that sensitive instructions are privileged, we make virtualisation much more efficient.

Problem: condition is not always satisfied

The Intel x86 instruction set, which is very popular, has 17 sensitive instructions that aren't privileged. This means these instructions can run in user mode without causing a trap to the operating system but still affect how the OS manages resources.

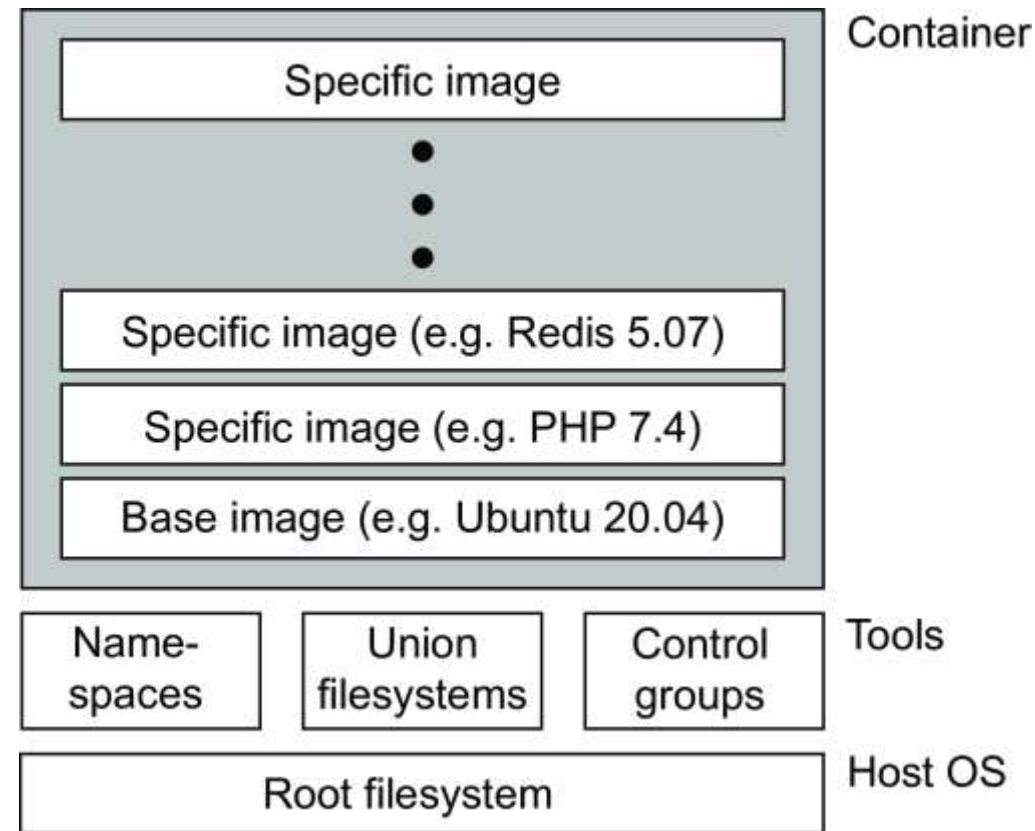
Solutions

- Emulate all instructions
- Wrap nonprivileged sensitive instructions to divert control to VMM
- Paravirtualisation: modify guest OS, either by preventing nonprivileged sensitive instructions, or making them nonsensitive (i.e., changing the context).

Containers

- Virtual machines let you run applications that need a specific operating environment, including its instruction set and operating system, across different platforms. However, many applications are stable with their instruction set and operating system but rely on specific libraries and support software. In these cases, we want different applications to run side-by-side, each with its own environment, without them noticing each other. This is where containers come in.
- Think of a container as a collection of binaries (also called images) that together make up the software environment for running applications. The easiest way to understand a container is to imagine what a user sees when logging into a Unix system: several standard directories containing executable programs, libraries, documentation, and so on.
- A simple way to implement a container would be to copy an entire environment for a specific use case and install it as a subdirectory of the root file system. An application inside this container would see exactly the libraries and dependencies it needs, while applications in other containers would see their own tailored environment.
- A container effectively virtualizes the software environment for an application, allowing it to run in its own isolated space even though it's sharing the same underlying system with other containers. This makes it much easier to manage and deploy applications without conflicts.
- However, this simple approach to containers isn't enough from a virtualisation perspective. First, applications and processes in different containers need to be isolated from each other. Second, just copying an entire environment for each container is pretty inefficient, especially since many libraries and resources are shared across containers. Lastly, the operating system that hosts the containers needs to control its resources effectively.

Containers



- Namespaces: a collection of processes in a container is given their own view of identifiers
- Union file system: combine several file systems into a layered fashion with only the highest layer allowing for write operations (and the one being part of a container).
- Control groups: resource restrictions can be imposed upon a collection of processes.

These mechanisms ensure that containers are efficient, isolated, and manageable.

Containers

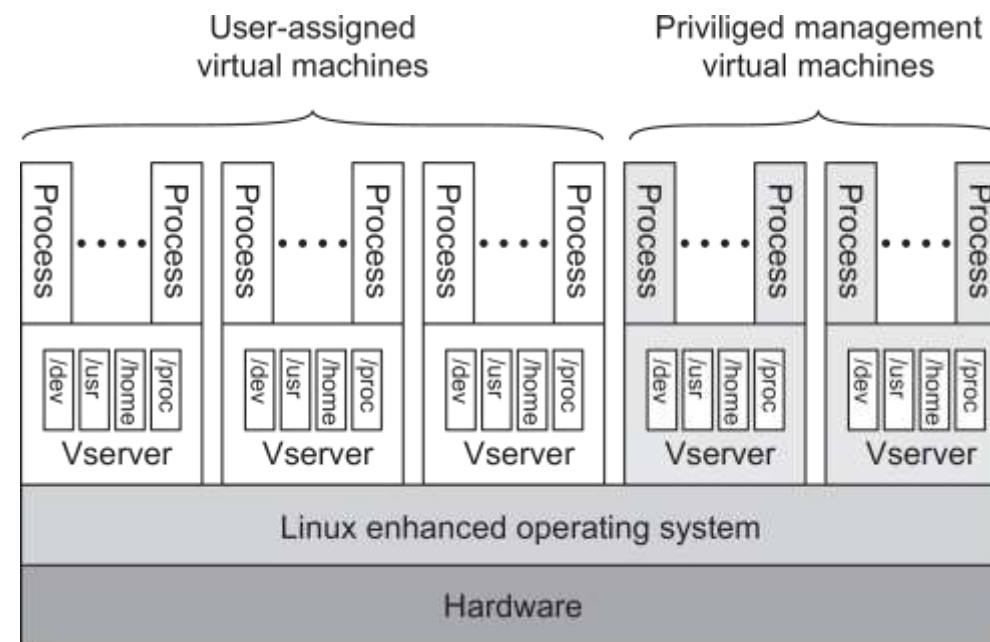
Example: PlanetLab

Essence: A collaborative distributed system with different organisations and each contribute one or more computers, adding up to a total of hundreds of nodes. These computers formed a server cluster where access, processing, and storage could all happen on each individual node.

Problem: We need to ensure that different distributed applications do not get into each other's way ⇒ virtualisation.

Containers

Example: PlanetLab basic organisation



Each participating organisation donated one or more computers (called nodes) to the project. These nodes were then shared among all PlanetLab users.

Two key components:

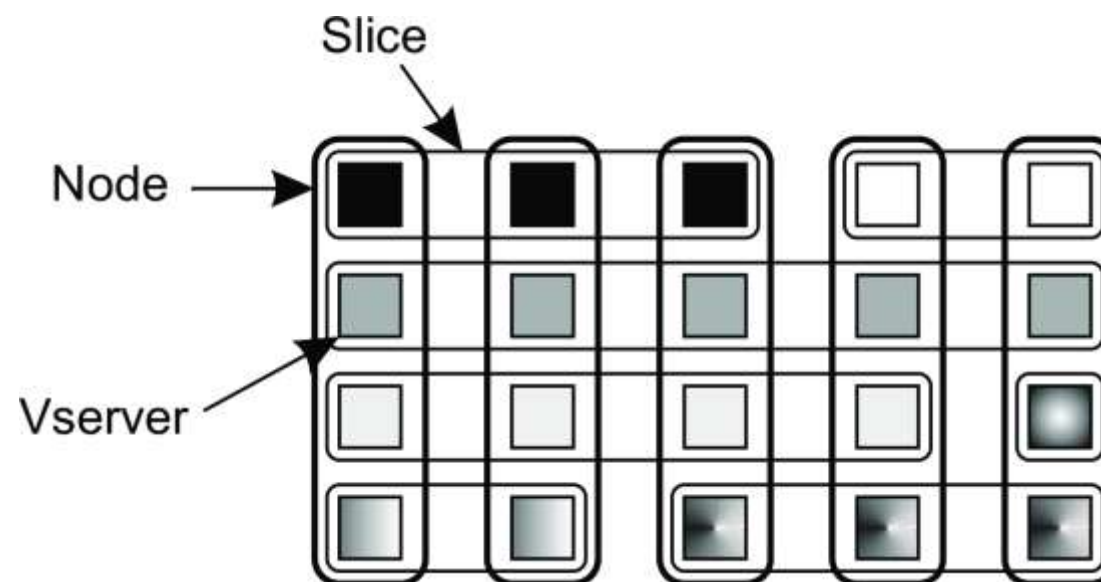
- Virtual Machine Monitor (VMM): This is an enhanced Linux operating system that supports containers. It's been adjusted to support the second component, which is Vserver.
- Vserver: These are essentially containers in execution. We can treat a Vserver as an independent and protected environment. Each Vserver has its own libraries, server versions, and other necessary software. This isolation means that different Vservers can run side-by-side on the same physical machine without interfering with each other. Distributed applications are assigned a collection of vservers distributed across multiple machines.

Containers

PlanetLab Vservers and slices

Essence

- The Linux VMM made sure that Vservers were kept separate. Processes in different Vservers ran at the same time and independently, each using only the software and programs available in their own environment. This isolation was strict. For example, two processes in different Vservers could have the same user ID, but that didn't mean they came from the same user. This separation made it much easier to support users from different organisations who wanted to use PlanetLab to experiment with different distributed systems and applications.
- To support such experimentation, PlanetLab used slices, each slice being a set of Vservers, each Vserver running on a different node. A slice can be thought of as a virtual server cluster, implemented by a collection of containers connected through a wide-area network.



Containers

PlanetLab Vservers and slices

Separation leads to slices

- The key part of managing PlanetLab resources was the node manager. Every node had its own manager, which was implemented as a separate Vserver. Its job was to create other Vservers on its node and manage resource allocation. To create a new slice, each node ran a slice creation service (SCS). The slice creation service could contact the node manager to request the creation of a Vserver and allocate resources. Importantly, the node manager couldn't be contacted directly over the network, so it could focus solely on local resource management.
- Keeping track of resources was done with a resource specification, which specified a time interval during which certain resources were allocated, including disk space, file descriptors, network bandwidth, memory, and CPU usage. A resource specification was identified by a globally unique 128-bit identifier known as a resource capability (rcap). Given an rcap, the node manager could look up the associated resource specification in a local table.
- Resources were bound to slices. In other words, to use resources, you needed to create a slice. Each slice was associated with a service provider, which is like having an account on PlanetLab.

Applying VM to Distributed Systems

VMs and Cloud Computing

Three Types of Cloud Services

- Infrastructure-as-a-Service covering the basic infrastructure
- Platform-as-a-Service covering system-level services
- Software-as-a-Service containing actual applications

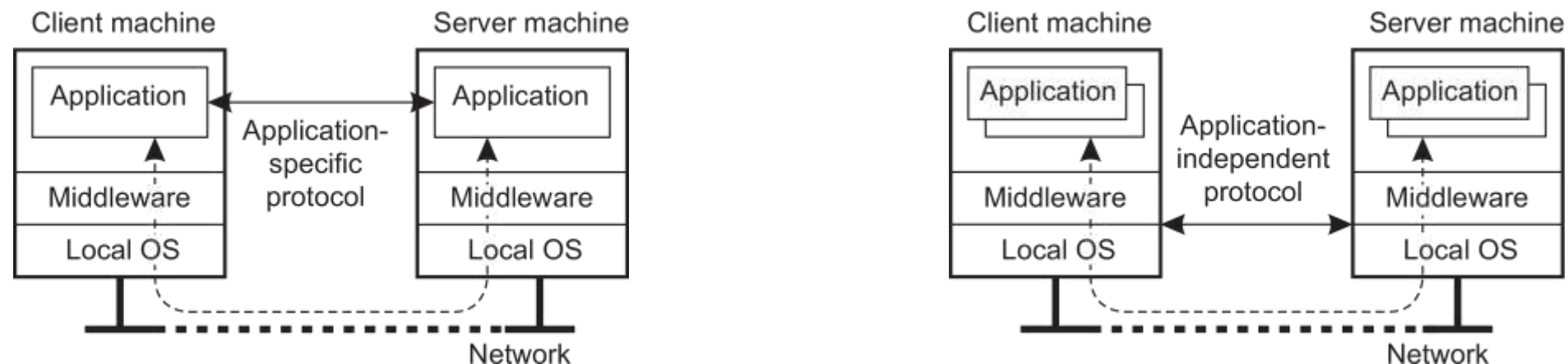
IaaS

Virtualisation is crucial for Infrastructure-as-a-Service. Instead of renting out a physical machine, a cloud provider rents out a virtual machine (VM). This VM might be sharing a physical machine with other customers. The great thing about virtualisation is that it creates the illusion for customers that they have their own dedicated physical machine, due to the almost complete isolation. However, this isolation isn't perfect because the physical resources are still shared, which can sometimes result in noticeable performance drops.

Clients

Client: Network User Interfaces

Client Server Interaction: Distinguish between application level and middleware-level solutions



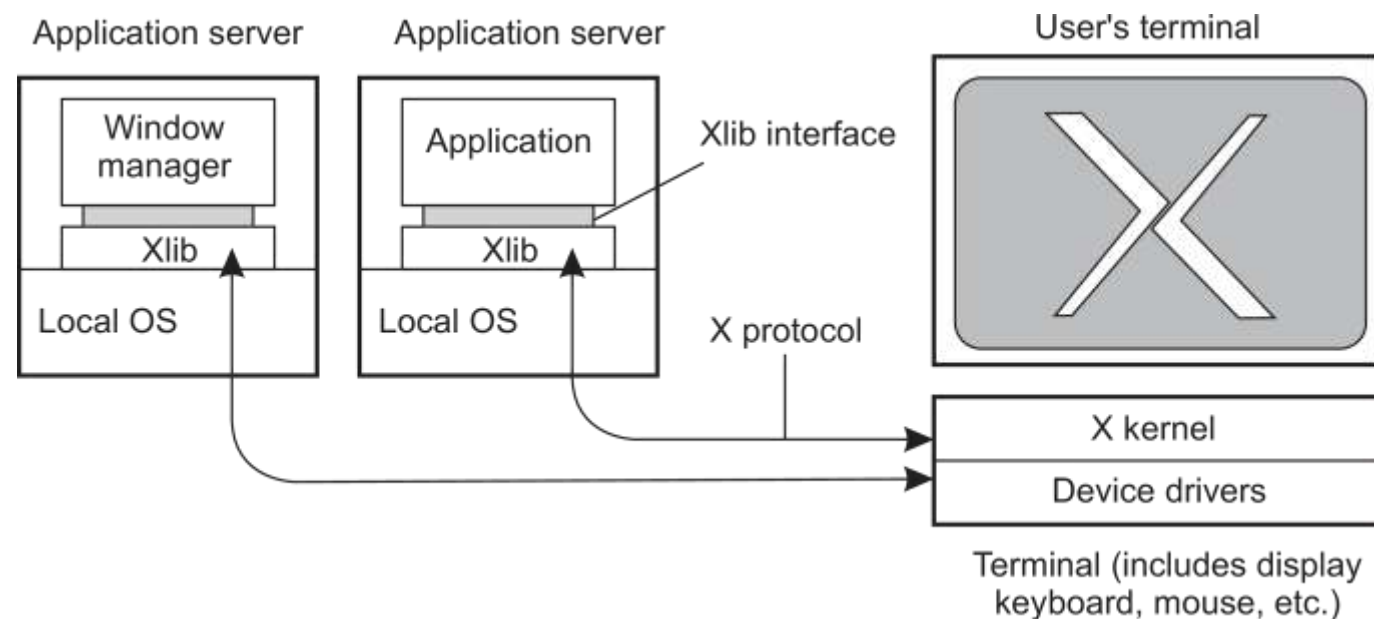
One of the main task of client machines is to help users interact with remote servers. There are roughly two ways to do this:

- First, for each remote service, the client machine will have a separate counterpart that can contact the service over the network. A typical example is a calendar running on a user's smartphone that needs to synchronize with a remote, possibly shared calendar. In this case, an application-level protocol will handle the synchronisation.
- A second solution is to provide direct access to remote services by offering only a convenient user interface. This means the client machine acts just like a terminal without needing local storage. This approach is application-neutral, as shown in right Figure. In networked user interfaces, everything is processed and stored on the server. This thin-client approach has become popular with the rise of Internet connectivity and mobile device usage. Thin-client solutions are also favored because they make system management much easier.

Client: Network User Interfaces

Example: The X Window System

Basic Organisation



X client and Server (1/2)

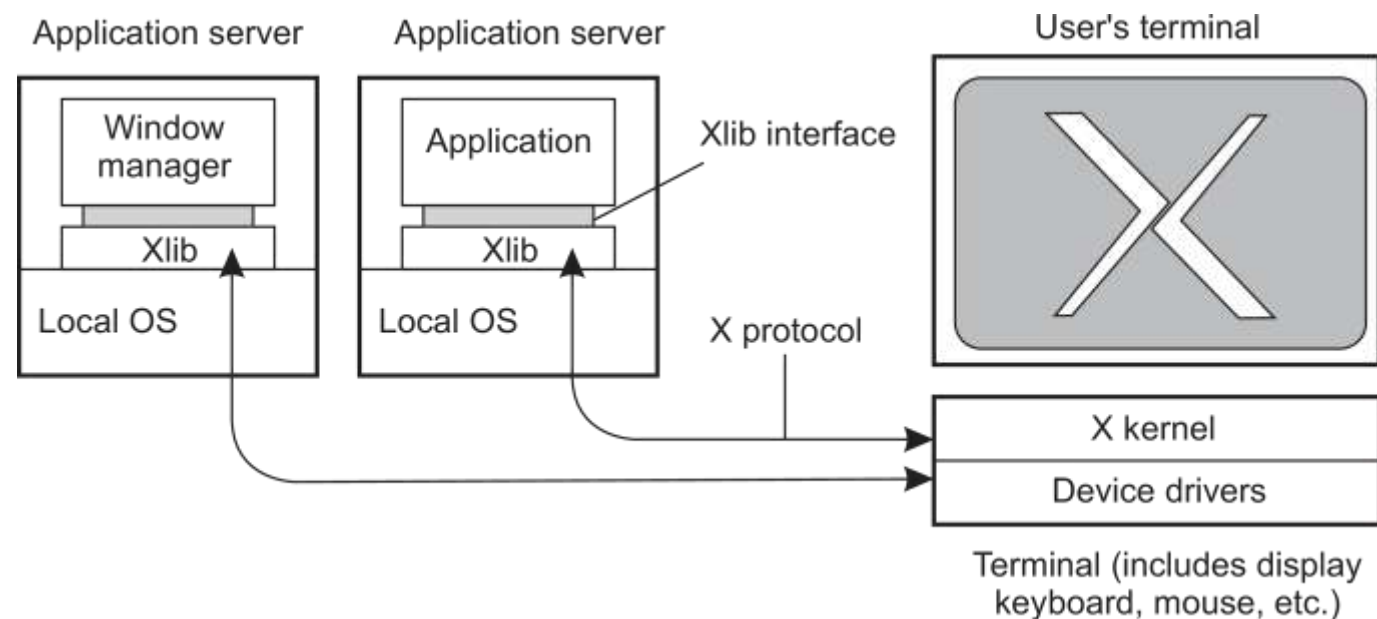
One of the oldest and still widely used networked user interfaces is the X Window System, often just called X. X is used to control bit-mapped terminals, which include a monitor, keyboard, and a pointing device like a mouse. Besides supporting traditional desktop computers and workstations, X also works with modern devices like touchscreens on tablets and smartphones. You can think of X as the part of an operating system that controls the terminal.

At the core of X is what we call the X kernel. This kernel contains all the terminal-specific device drivers and is generally highly hardware dependent.

Client: Network User Interfaces

Example: The X Window System

Basic Organisation



X client and Server (2/2)

The X kernel offers a low-level interface for controlling the screen and capturing events from the keyboard and mouse. This interface is made available to applications through a library called Xlib.

What's interesting about X is that the X kernel and X applications don't have to be on the same machine. X provides the X protocol, an application-level communication protocol that lets an instance of Xlib exchange data and events with an X kernel. For example, Xlib can send requests to the X kernel to create or close a window, set colors, and define the type of cursor to display. In turn, the X kernel responds to local events like keyboard and mouse input by sending event packets back to Xlib.

Client: Network User Interfaces

Improving X: Practical Observations

- Applications use specific display commands provided by X to manipulate the display. These commands are usually sent over the network and executed by the X kernel. Ideally, applications written for X should separate the application logic from the user-interface commands. Unfortunately, this often doesn't happen.
- Much of the application logic and user interaction are tightly linked, meaning an application will send many requests to the X kernel and wait for a response before moving on. This synchronous behavior can hurt performance, especially over wide-area networks with long delays.

Client: Network User Interfaces

Improving X: Alternative Approaches

- One is to redesign the X protocol, like with NX. NX Client is a computer program that allows your PC to act like a terminal to talk to a remote unix machine. A big part of this work focuses on reducing bandwidth by reducing the size of X messages. Messages are split into a fixed part (an identifier) and a variable part. Often, multiple messages have the same identifier and similar data. This can be used to send only the differences between these messages. By keeping track of identifiers, the receiver can easily decode the messages. This approach can reduce bandwidth usage by up to 1000 times, allowing X to work over low-bandwidth links.
- Let the application control the remote display down to the pixel level, also known as controlling a remote desktop. Changes in the display bitmap are sent over the network to the display, where they are updated in the local frame buffer. A well-known example of this is Virtual Network Computing (VNC). Letting the application control the display requires sophisticated encoding techniques to prevent bandwidth issues.

Client: Virtual Desktop Environment

Logical Development

With the development of cloud computing and more cloud applications appeared, it made sense to turn the cloud into a virtual desktop environment for users. All users needed was some client-side software to access that desktop. Google was one of the first to do this with Chrome OS.

- Idea : let the browser provide the local desktop interface. Besides the browser, there are multiple stand-alone applications, each of which eventually connects with a cloud-based counterpart. In a way, this setup is similar to what modern smartphones offer.
- With the trend of transforming applications into browser extensions, we see the browser taking over the role of the operating system's user interface. So, the browser is becoming more like a desktop environment where users can run different applications that are connected to the cloud.



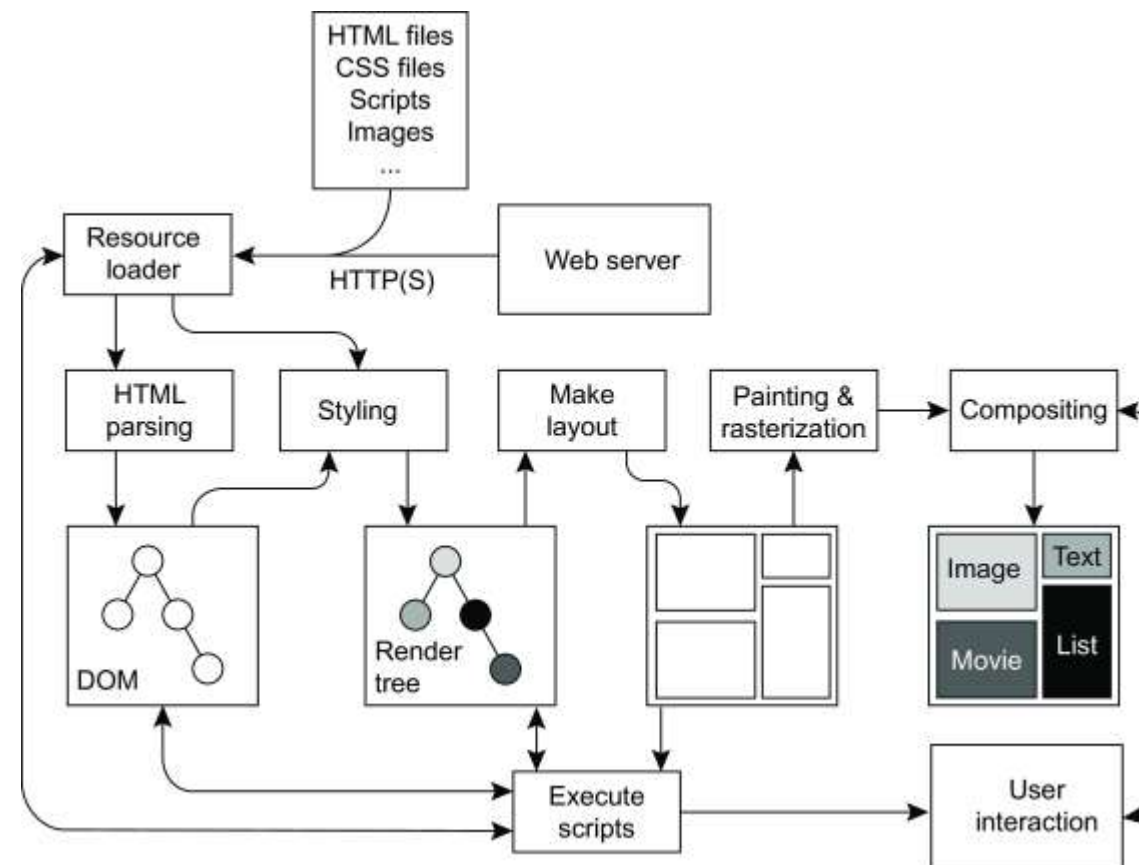
The Google Chromebook

Client: Virtual Desktop Environment

The Anatomy of a Web Browser (1/5)

The Chrome browser is a complex piece of software with over 25 million lines of code, similar to the size of the Linux kernel. Back when the Web started, it was just simple HTML pages. Now, we have not only advanced markup language but also a bunch of tools for interaction and client-side scripting languages.

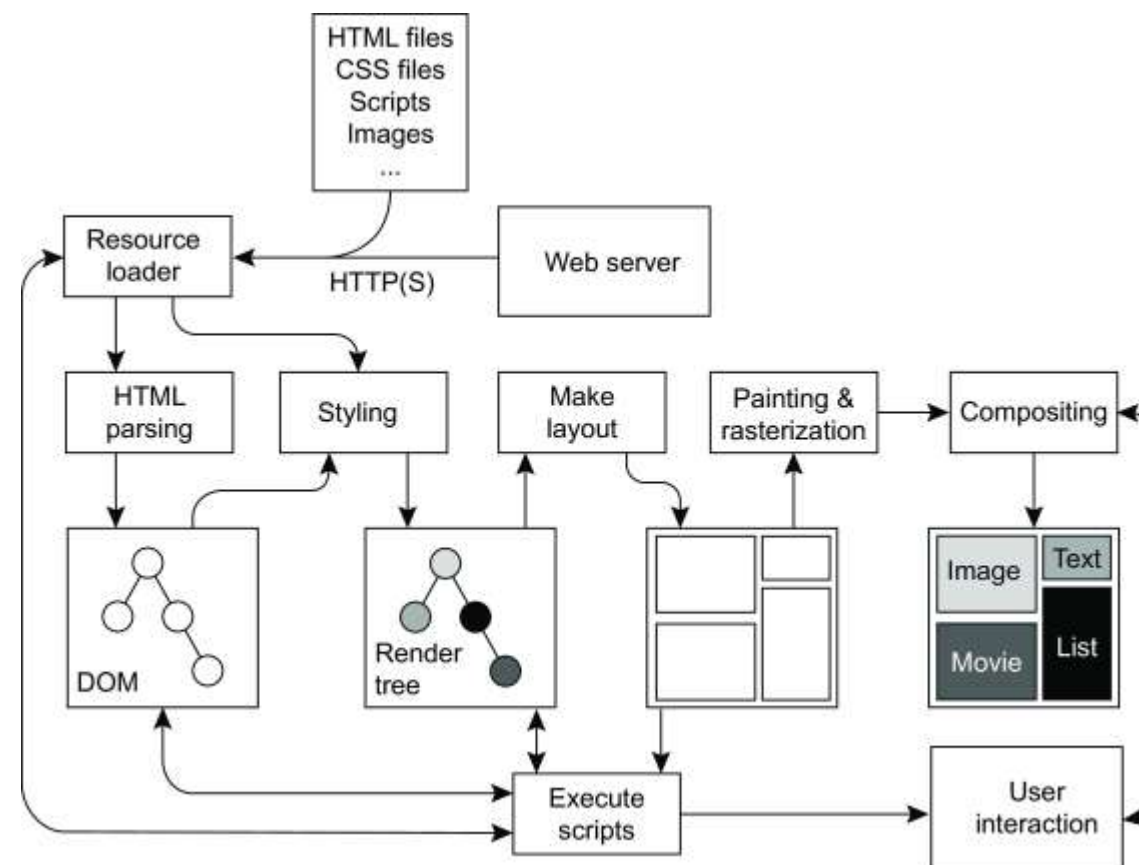
At the core of Chrome, we have the resource loader, which fetches content from a web server. It usually starts by fetching an HTML file and then, in parallel, sets up connections for other content referenced in that page using separate threads. The resource loader needs to partially analyze the HTML file to figure out what other content it needs to fetch. Most of the analysis is done by another component that builds something called a Document Object Model (DOM). A DOM is basically a tree representation of the HTML file.



Client: Virtual Desktop Environment

The Anatomy of a Web Browser (2/5)

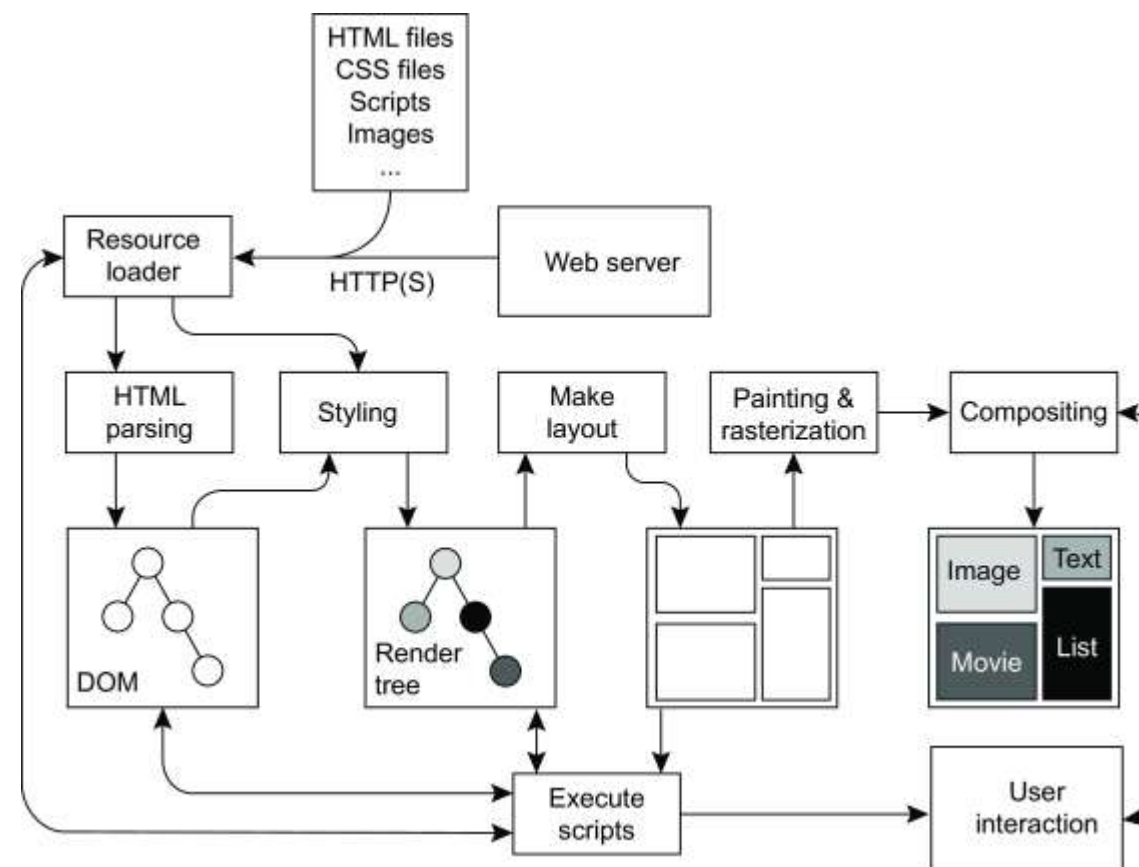
The DOM represents the structure of the page, but the actual styling information comes from a separate document. For example, it might say that all paragraphs should be in a specific color and font. This styling information is analyzed separately and added to the DOM, creating a render tree. This tree has all the information needed for the next step: determining where each element will be displayed. This involves calculating geometric regions for different parts of the DOM and deciding where lines will break, which can get complex with elements like floating images or multiple columns.



Client: Virtual Desktop Environment

The Anatomy of a Web Browser (3/5)

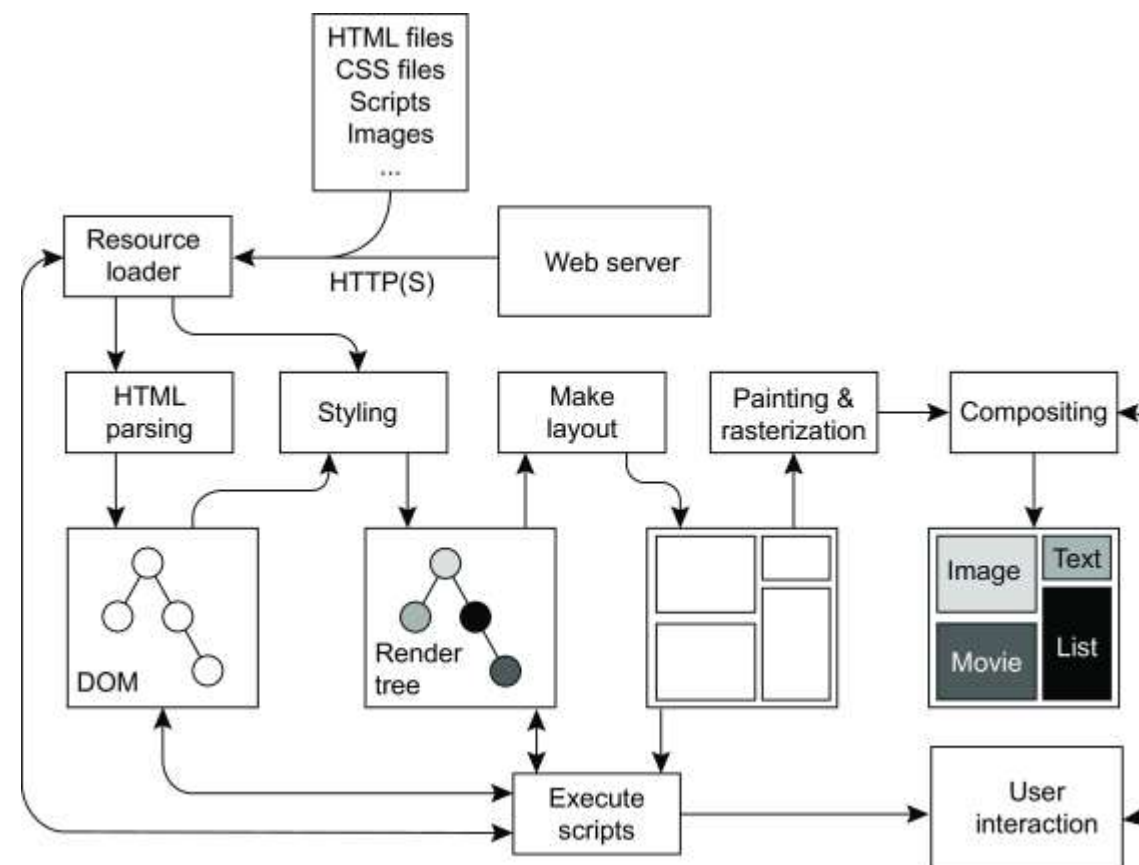
Once the layout is determined, the painting component takes over, creating a program of paint operations, like `drawRectangle(x, y, height, width)`. These operations are executed by the rasterisation process, which fills in all the details for every pixel on the screen, including colors. Rasterisation also handles embedded images, ensuring each pixel of an image is displayed correctly. The DOM is actually broken down into several layers, each rasterised separately and then composed into the final image displayed on the screen. The compositor handles user interactions like scrolling through the page.



Client: Virtual Desktop Environment

The Anatomy of a Web Browser (4/5)

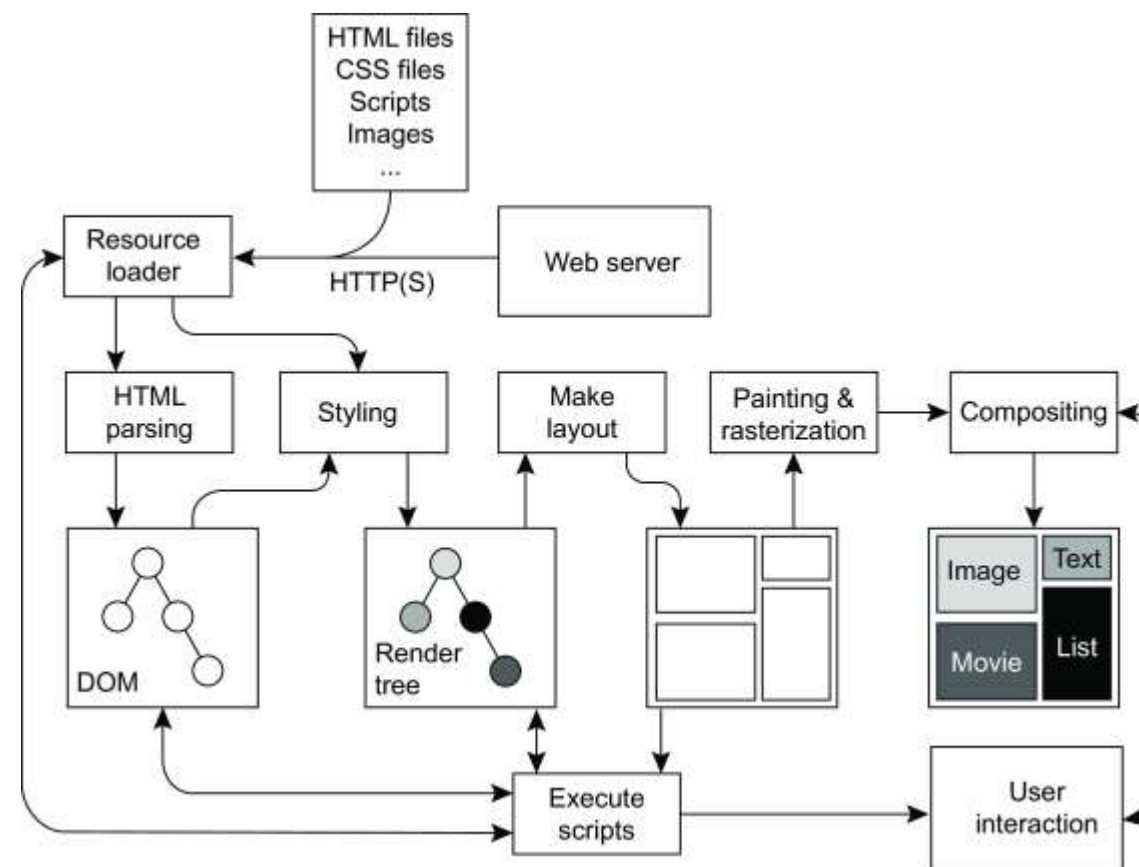
Modern browsers also have a component for handling scripts, like JavaScript. JavaScript is a popular scripting language for client-side web applications. The code runs with the browser's permissions, making it relatively safe. Running code inside the browser is powerful because it makes the user feel like they're running an application on their local desktop.



Client: Virtual Desktop Environment

The Anatomy of a Web Browser (5/5)

Rendering a web page is a highly complex process, with many parts like rasterising layers done in parallel, sometimes using special processors like a GPU. Modern web browsers use threads and processes not only for better structure and organisation but also for efficient rendering. For example, in Chrome, the rendering is handled by a separate process that includes HTML parsing, styling, layout making, painting, and compositing. Rasterisation might use a GPU and communicates with the main process through message passing. Each browser tab has its own renderer process, and to improve security, each renderer runs in a separate sandbox to prevent direct communication with the underlying operating system.



Client-side Software

Generally Tailored for Distribution Transparency (1/2)

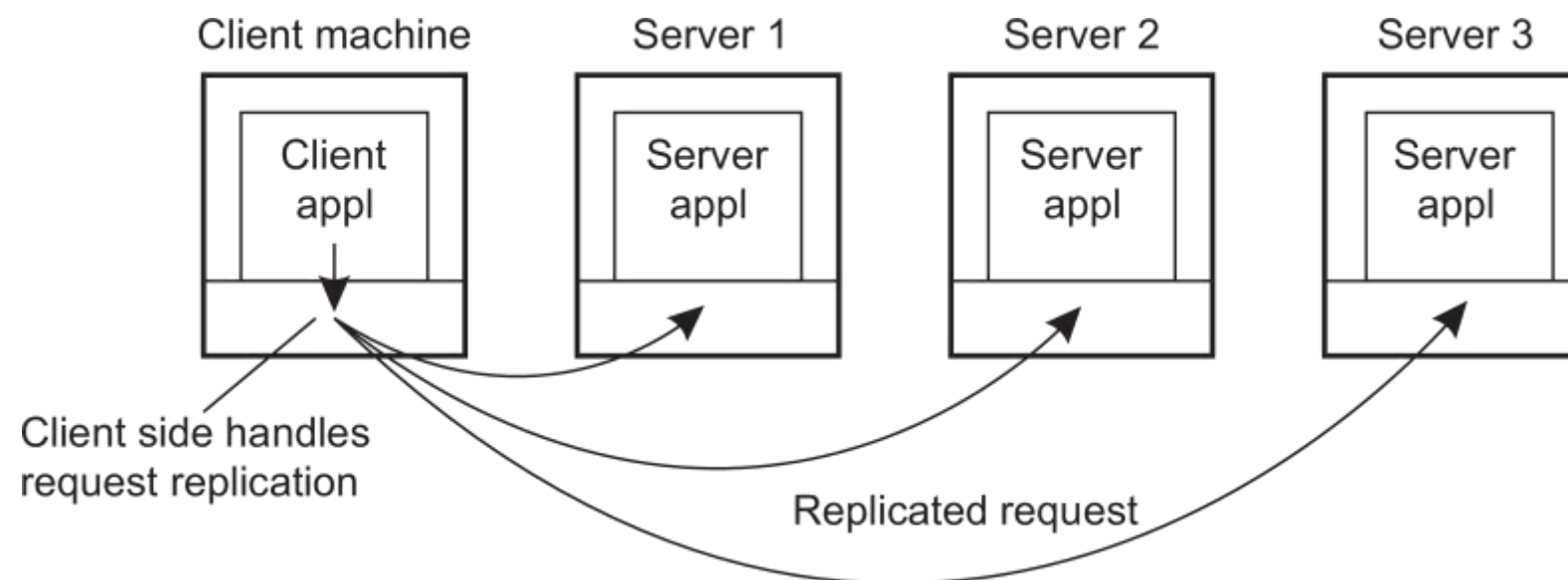
Client software includes more than just user interfaces. Often, parts of the processing and data in a client-server application are handled on the client side too. A special class of client software is found in embedded systems like ATMs, cash registers, barcode readers, and TV set-top boxes. In these cases, the user interface is a small part of the software compared to the local processing and communication functions. Besides the user interface and application-related software, client software includes components to achieve distribution transparency. Ideally, a client shouldn't know it's communicating with remote processes. On the other hand, distribution is often less transparent to servers for performance and correctness reasons.

- Access transparency: It generally handled by generating a client stub from an interface definition of what the server offers. The stub provides the same interface as the server but hides differences in machine architectures and communication. The client stub converts local calls to messages sent to the server and vice versa, transforming server messages to return values as if calling a regular procedure.
- Location/migration transparency: There are many ways to handle location, migration, and relocation transparency. Using a good naming system is crucial. Often, cooperation with client-side software is important. For example, when a client is already connected to a server, it can be informed directly if the server changes location. In this case, the client's middleware can hide the server's new network location and rebind to the server if necessary. At worst, the client application might notice a temporary drop in performance.
- Replication transparency: Similarly, many distributed systems implement replication transparency using client-side solutions. For instance, in a distributed system with replicated servers, requests can be forwarded to each replica. The client-side software can then collect all responses and pass a single response to the client application.
- Failure transparency: Client middleware usually handles masking communication failures with a server. For example, it can repeatedly attempt to connect to a server or try another server after several attempts. Sometimes, the client middleware returns data it cached during a previous session, like how web browsers work when they can't connect to a server.

Client-side Software

Generally Tailored for Distribution Transparency (2/2)

Client software does a lot more than just handle the user interface. It also plays a crucial role in processing, communication, and maintaining transparency in distributed systems.



Servers

Servers: General Organisation

Basic Model

A server is a process implementing a specific service on behalf of a collection of clients. It waits for an incoming request from a client and subsequently ensures that the request is taken care of, after which it waits for the next incoming request.

Two basic types

- Iterative server: The server itself handles the request and, if necessary, returns a response to the requesting client.
- Concurrent server: It does not handle the request itself, but passes it to a separate thread or another process, after which it immediately waits for the next incoming request. A multithreaded server is an example of a concurrent server. An alternative implementation of a concurrent server is to fork a new process for each new incoming request. This approach is followed in many Unix systems. The thread or process that handles the request is responsible for returning a response to the requesting client.

Observation

Concurrent servers have become the standard because they can easily handle multiple requests at the same time. This is especially useful when dealing with operations that might block, like accessing a disk or communicating with other servers.

Servers: General Organisation

Contacting a Server (1/2)

In all cases, clients send requests to an end point, also called a port, on the server's machine. Each server listens for requests on a specific end point. But how do clients know the end point of a service?

Observation: most services are tied to a specific port

For example, servers handling Internet FTP requests always listen on TCP port 21, and HTTP servers for the web always listen on TCP port 80. These ports are assigned by the Internet Assigned Numbers Authority (IANA). With these assigned end points, the client only needs to know the network address of the server's machine. Name services can help find that address.

ftp-data	20	File Transfer [Default Data]
ftp	21	File Transfer [Control]
telnet	23	Telnet
smtp	25	Simple Mail Transfer
www	80	Web (HTTP)

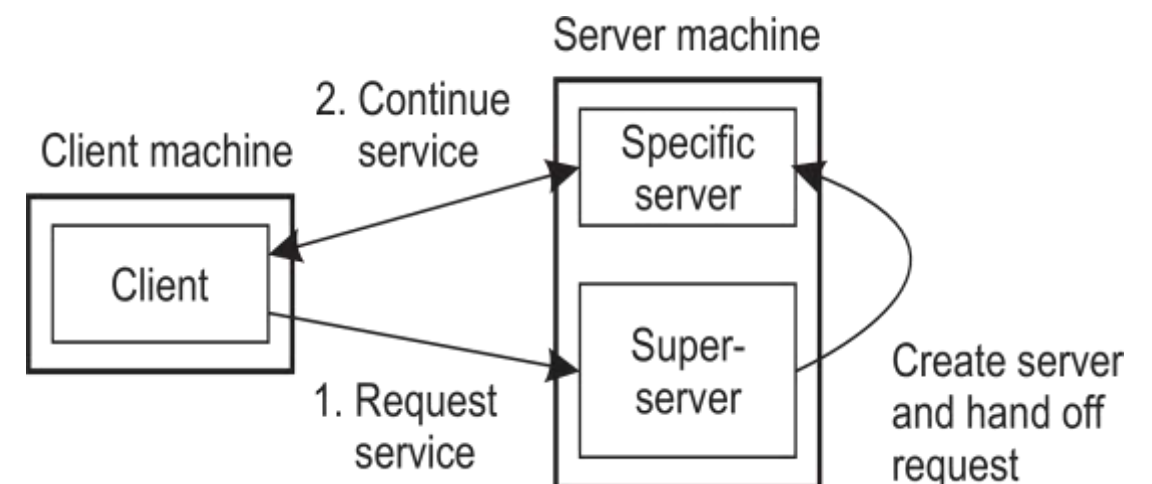
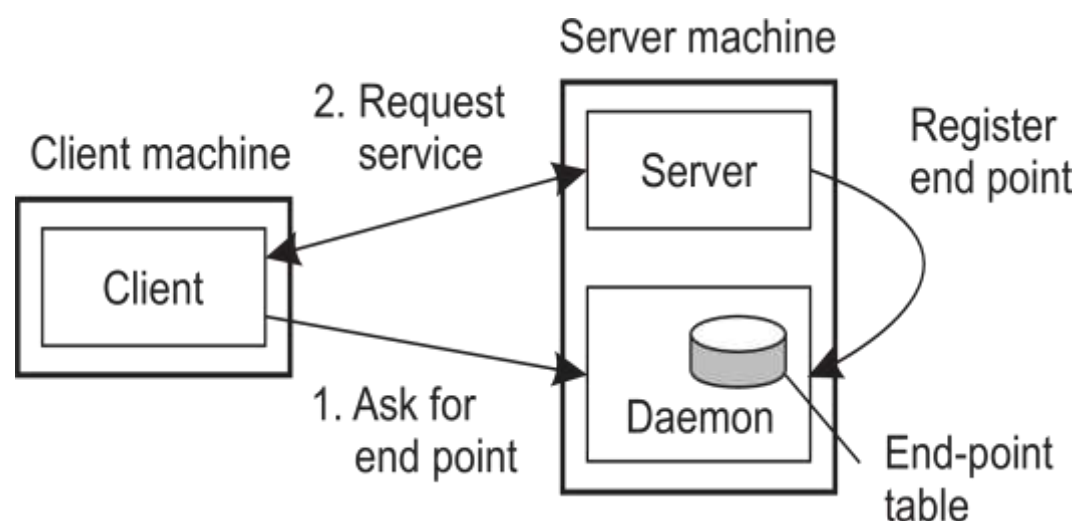
Servers: General Organisation

Contacting a Server (2/2)

Dynamically assigning an end point: two approaches

However, not all services need a preassigned end point. For instance, a time-of-day server might use an end point dynamically assigned by its local operating system. In this case, a client first has to look up the end point. One solution is to have a special daemon running on each machine with servers. This daemon keeps track of the current end point of each service implemented by a co-located server. The daemon itself listens on a well-known end point. So, a client first contacts the daemon to request the end point and then contacts the specific server.

It's common to associate an end point with a specific service. However, having a separate server for each service can waste resources. For example, in a typical Unix system, many servers run simultaneously, most passively waiting for a client request. Instead of keeping track of many passive processes, it can be more efficient to have a single superserver listening to each end point associated with a specific service.



Servers: General Organisation

Out-of-band communication: Issue

When designing a server is how to handle interruptions. For example, imagine a user starts uploading a huge file to an FTP server but then realizes it's the wrong file and wants to cancel the upload. There are a few ways to do this.

Solution 1: Use a separate port for urgent data

- Server has a separate thread/process for urgent messages, the server listen to a separate control end point for out-of-band data while also listening (with a lower priority) to the end point for normal data.
- Urgent message comes in $\Rightarrow$ associated request is put on hold
- Note: we require OS supports priority-based scheduling

Solution 2: Use facilities of the transport layer

- Send out-of-band data over the same connection used for the original request
- Example: TCP allows for urgent messages in same connection
- Urgent messages can be caught using OS signaling techniques

Servers: General Organisation

Servers and State: Stateless servers (1/2)

- A stateless server doesn't keep information about its clients' states and can change its own state without having to inform any client. For example, a web server is stateless. It just responds to incoming HTTP requests, like uploading a file or, more commonly, fetching a file. Once it processes the request, the web server forgets about the client. The collection of files managed by the web server can be changed without needing to inform the clients.
- In many stateless designs, the server actually does maintain some information about its clients. However, the key point is that if this information is lost, it won't disrupt the service offered by the server. For instance, a web server typically logs all client requests. This log is useful for deciding whether to replicate certain documents and where to replicate them. If the log is lost, the only consequence is possibly suboptimal performance, but the service continues.
- A special form of stateless design is where the server maintains what is known as "soft state." In this case, the server promises to keep some state information on behalf of the client, but only for a limited time. After that time, the server discards the information and returns to its default behavior. For example, a server might promise to keep a client informed about updates but only for a limited time. After that, the client has to poll the server for updates. Soft-state approaches come from protocol design in computer networks but can also be applied to server design.

Servers: General Organisation

Servers and State: Stateless servers (1/2)

Never keep accurate information about the status of a client after having handled a request:

- Don't record whether a file has been opened (simply close it again after access)
- Don't promise to invalidate a client's cache
- Don't keep track of your clients

Consequences

- Clients and servers are completely independent
- State inconsistencies due to client or server crashes are reduced
- Possible loss of performance because, e.g., a server cannot anticipate client behavior (think of prefetching file blocks)

Servers: General Organisation

Servers and State

Question

Does connection-oriented communication fit into a stateless design?

Servers: General Organisation

Servers and State

Question

Does connection-oriented communication fit into a stateless design?

Answer

Connection-oriented communication, like using TCP, means that there's a continuous connection between the client and the server. This connection inherently carries some state information, such as the status of the connection and the data being transferred.

In a true stateless design, the server doesn't keep any information about the client after handling a request. This means the server shouldn't remember anything about the client or the connection once the request is processed. But with connection-oriented communication, the server needs to keep track of the connection state, which seems to contradict the idea of being stateless.

However, we can make it work in a semi-stateless way. The server can treat each request independently, without relying on previous interactions, even if there's a continuous connection. It doesn't record details like whether a file was opened or make promises to invalidate a client's cache. The server just processes each request as if it's the first time it's seeing it.

While pure statelessness and connection-oriented communication might not perfectly align, we can design the server to minimize state dependencies and handle requests independently, which keeps us pretty close to the stateless model.

Servers: General Organisation

Servers and State: Stateful Servers

Keeps track of the status of its clients:

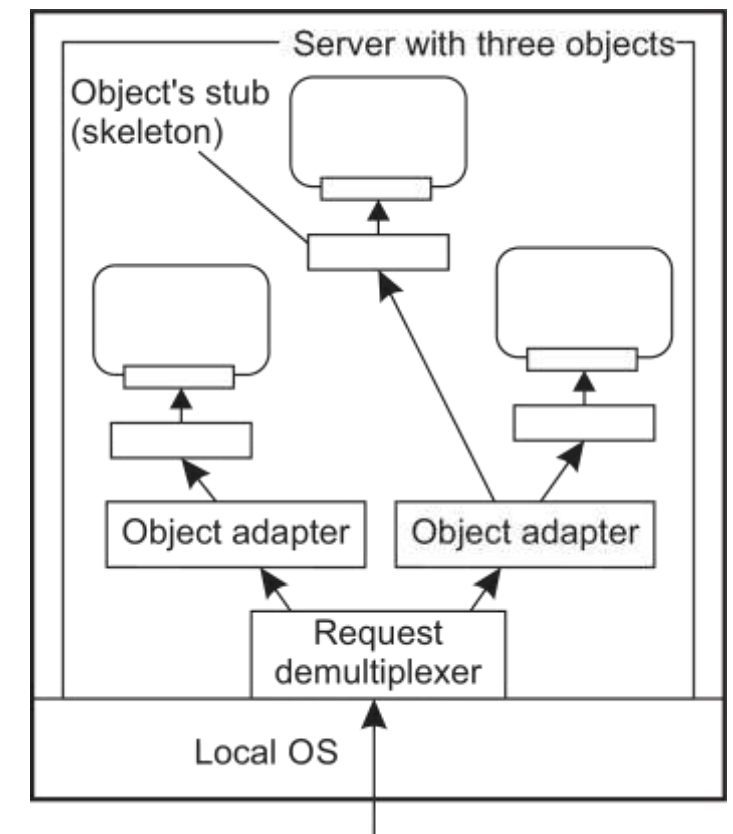
- The server maintains some persistent information and needs to explicitly delete it when it's no longer needed. For example, consider a file server that lets a client keep a local copy of a file for updates. This server would keep a table of (client, file) entries to track which client has update permissions on which file and possibly the most recent version of that file.
- Record that a file has been opened, so that prefetching can be done
- Knows which data a client has cached, and allows clients to keep local copies of shared data

Observation

This approach can improve the performance of read and write operations from the client's perspective. Performance improvement is often a significant benefit of stateful designs. However, the downside is that if the server crashes, it needs to recover its table of (client, file) entries to ensure it has processed the most recent updates on a file. Generally, a stateful server must recover its entire state as it was just before the crash, which can introduce a lot of complexity. In a stateless design, no special measures are needed for recovery. The server simply restarts and waits for client requests to come in.

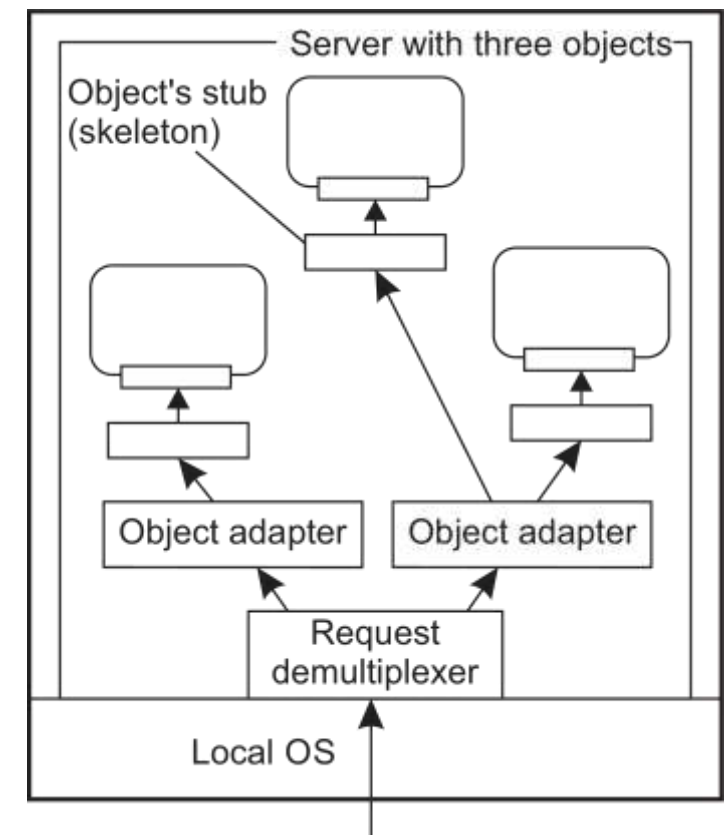
Object Servers (1/2)

- Unlike traditional servers, an object server doesn't offer a specific service on its own. Instead, it hosts objects that provide various services. So, the server's main job is to manage these objects and handle requests from remote clients. This setup makes it easy to add or remove services by simply adding or removing objects.
- Think of an object server as a place where objects "live." An object has two main parts: the data that represents its state and the code that defines its methods. How these parts are managed can be different. In some servers, each object might have its own thread, while in others, a new thread might be used for each request to the object.
- Activation policy: which actions to take when an invocation request comes in:
 - Where are code and data of the object?
 - Which threading model to use?
 - Keep modified state of object, if any?



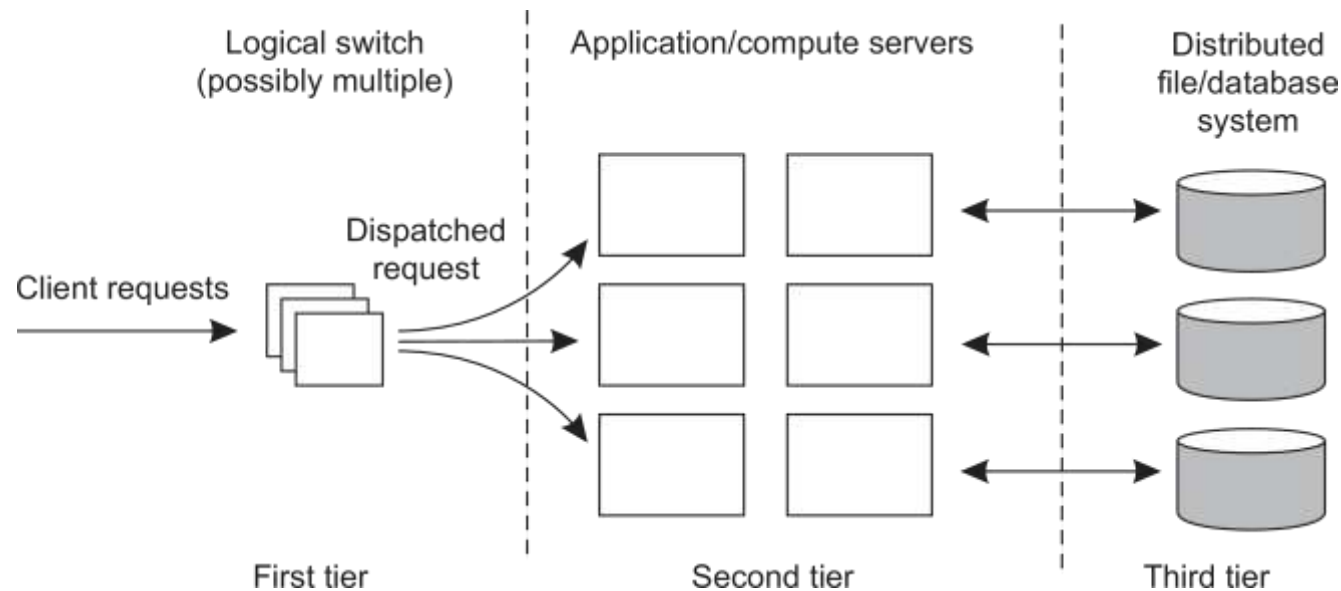
Object Servers (2/2)

- Object adapter: To manage activation policies, servers use something called an object adapter. Think of an object adapter as a piece of software that enforces a specific activation policy. These adapters are generic tools that developers can configure based on the needs of their objects.
 - Object adapter control one or multiple objects. Because a server might need to support objects with different activation policies at the same time, it can have several adapters. When a request comes in, it's directed to the appropriate adapter.
 - One key point about object adapters is that they don't need to know the specific interfaces of the objects they manage. This makes them flexible and generic. The adapter's job is to extract the object reference from the request and then pass it on to the correct object.
 - Instead of sending the request directly to the object, the adapter hands it off to a server-side stub, or skeleton, of that object. The stub is created based on the object's interface definitions, and it processes the request before invoking the appropriate method on the object.
 - the use of object adapters allows for a flexible and efficient way to manage and invoke objects in a distributed system, making it easier to handle a variety of services dynamically.



Server Clusters

Three Different Tiers: Common Organisation (1/2)



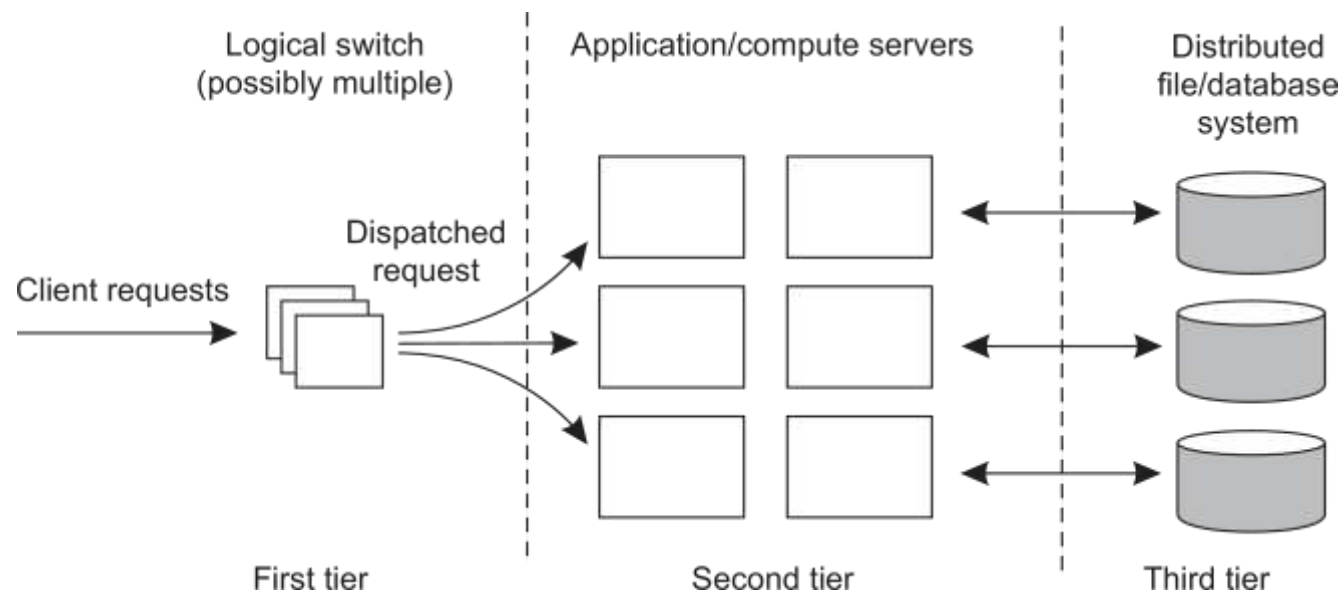
Crucial element

The first tier is a logical switch that routes client requests. This switch can take many forms. For example, transport-layer switches accept incoming TCP connection requests and forward them to one of the servers in the cluster. Another example is a Web server that handles incoming HTTP requests, passes some of them to application servers for further processing, collects the results, and then sends back an HTTP response.

In many server clusters, especially in enterprise setups, you'll find servers dedicated to application processing in the second tier. These servers might not need to be high-performance machines if the bottleneck is access to storage rather than compute power. However, in cluster computing scenarios that require a lot of compute power, these application servers are often high-performance machines designed to handle intensive processing tasks.

Server Clusters

Three Different Tiers: Common Organisation (2/2)



Crucial element

The third tier consists of data-processing servers like file and database servers. These servers may run on specialized machines optimized for high-speed disk access and large server-side data caches, depending on the cluster's needs.

Not all server clusters stick to this three-tier structure. Sometimes, each machine in the cluster has its own local storage, combining application and data processing into a single server. This setup creates a two-tier architecture. For example, in a server cluster used for streaming media, it's common to see a two-tier system where each machine acts as a dedicated media server, handling both application and data processing tasks

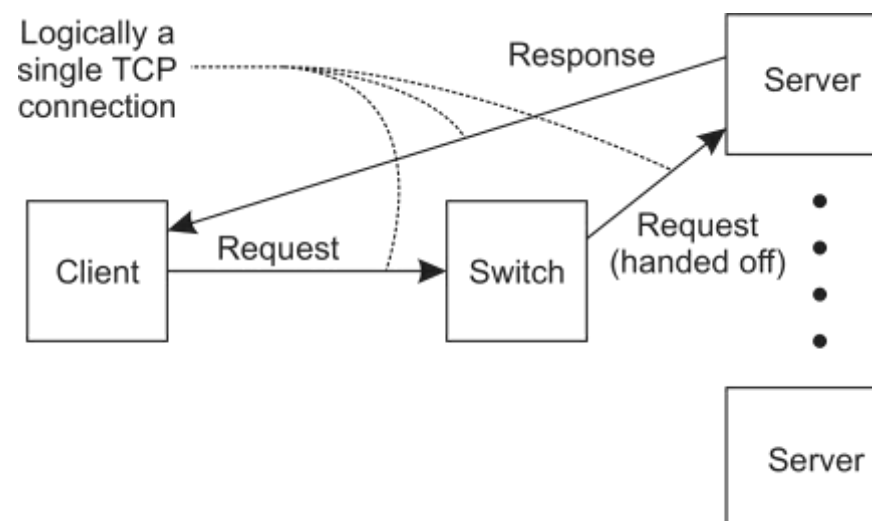
Server Clusters

Request Handler (1/3)

Observation

First tier usually handle communications. Having the first tier handle all communication from/to the cluster may lead to a bottleneck.

Solution TCP handoff - A switch can actually hand off the connection to a selected server such that all responses are directly communicated to the client without passing through the switch.

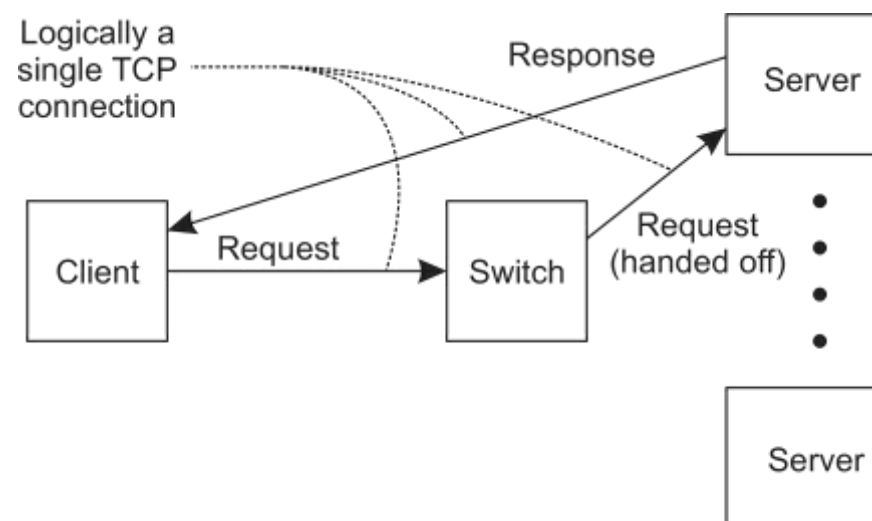


Server Clusters

Request Handler (2/3)

Solution

TCP handoff - When the switch receives a TCP connection request, it first identifies the best server for handling that request, and forwards the request packet to that server. The server, in turn, will send an acknowledgment back to the requesting client, but inserting the switch's IP address as the source field of the header of the IP packet carrying the TCP segment. Note that this address rewriting is necessary for the client to continue executing the TCP protocol: it is expecting an answer back from the switch, not from some arbitrary server it has never heard of before. Clearly, a TCP-handoff implementation requires operating-system level modifications. TCP handoff is especially effective when responses are much larger than requests, as in the case of Web servers.

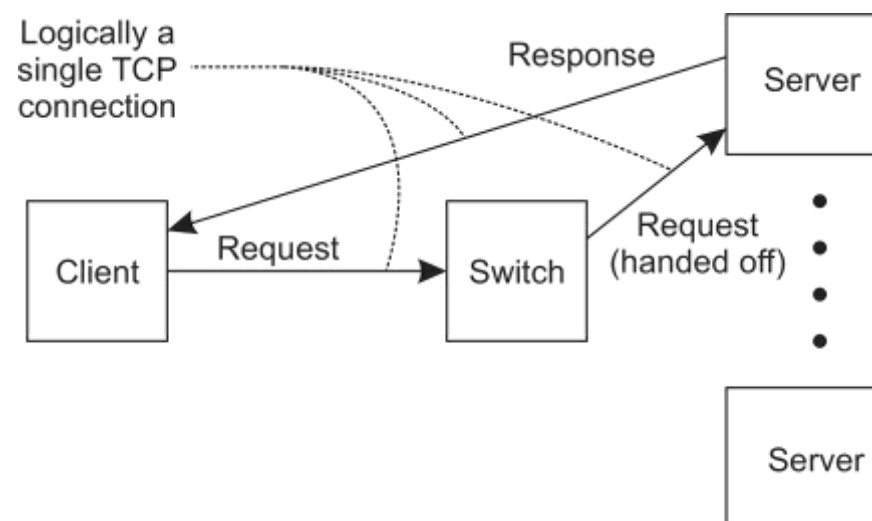


Server Clusters

Request Handler (3/3)

Solution

TCP handoff - It can be seen that the switch can play an important role in distributing the load among the various servers. By deciding where to forward a request to, the switch also decides which server is to handle further processing of the request. The simplest load-balancing policy that the switch can follow is round robin: each time it picks the next server from its list to forward a request to. Of course, the switch will have to keep track to which server it handed off a TCP connection, at least until that connection is torn down. As it turns out, maintaining this state and handing off subsequent TCP segments belonging to the same TCP connection, may actually slow down the switch.



Server Clusters

When Servers are Spread Across the Internet

Observation

Spreading servers across the Internet may introduce administrative problems. These can be largely circumvented by using data centers from a single cloud provider.

Request Dispatching: if local is important

Common Approach: us DNS

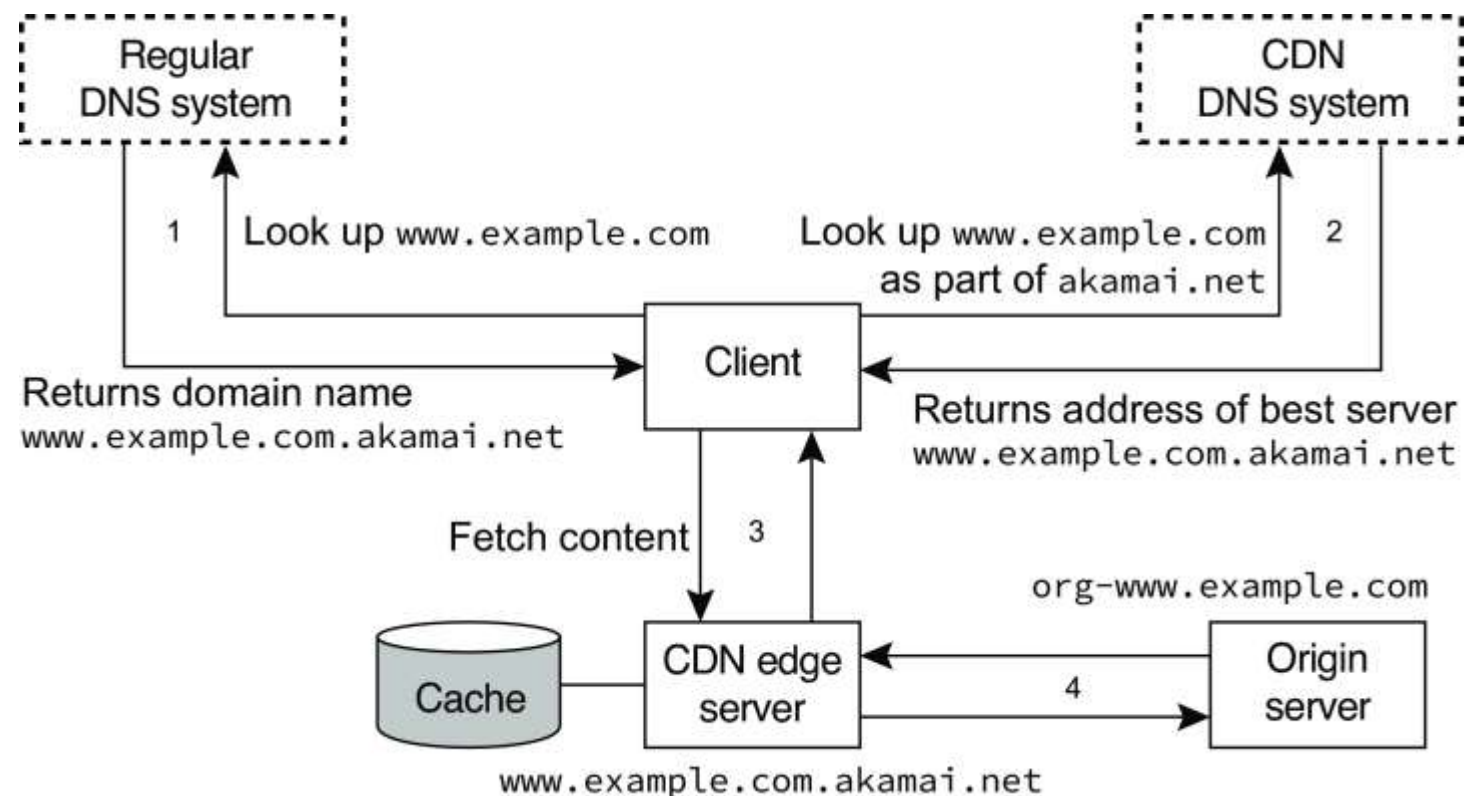
1. Client looks up specific service through DNS - client's IP address is part of request
2. DNS server keeps track of replica servers for the requested service, and returns address of most local server.

Client Transparency

To keep client unaware of distribution, let DNS resolver act on behalf of client. Problem is that the resolver may actually be far from local to the actual client.

Server Clusters

A Simplified Version of the Akamai CDN (1/5)

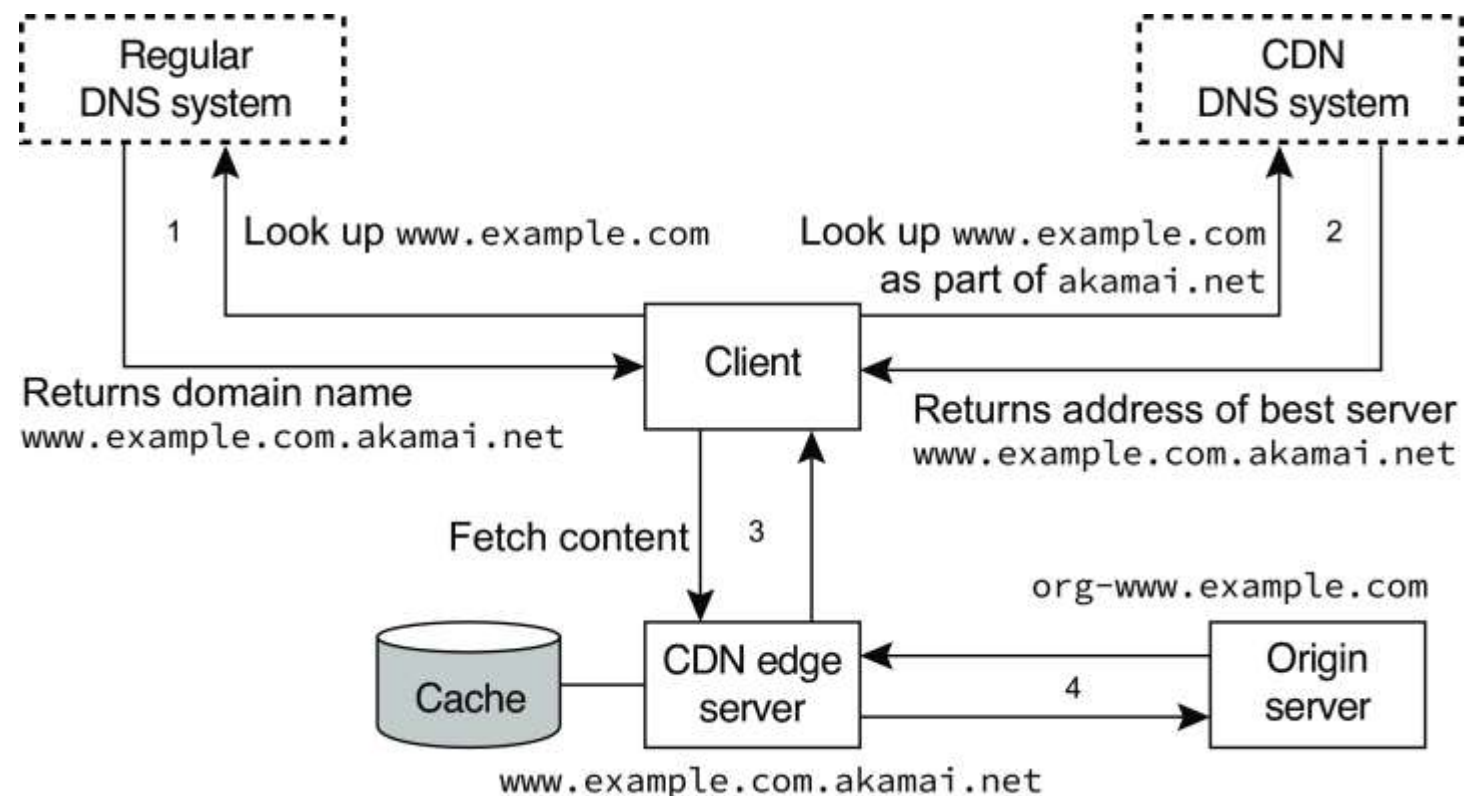


Important Note

The closer a video server is located to a client, the easier it becomes to provide high-quality streams. This approach is followed by so-called cloud-based Content Delivery Networks. CDNs form an important group of distributed systems that make use of wide-area clusters.

Server Clusters

A Simplified Version of the Akamai CDN (2/5)

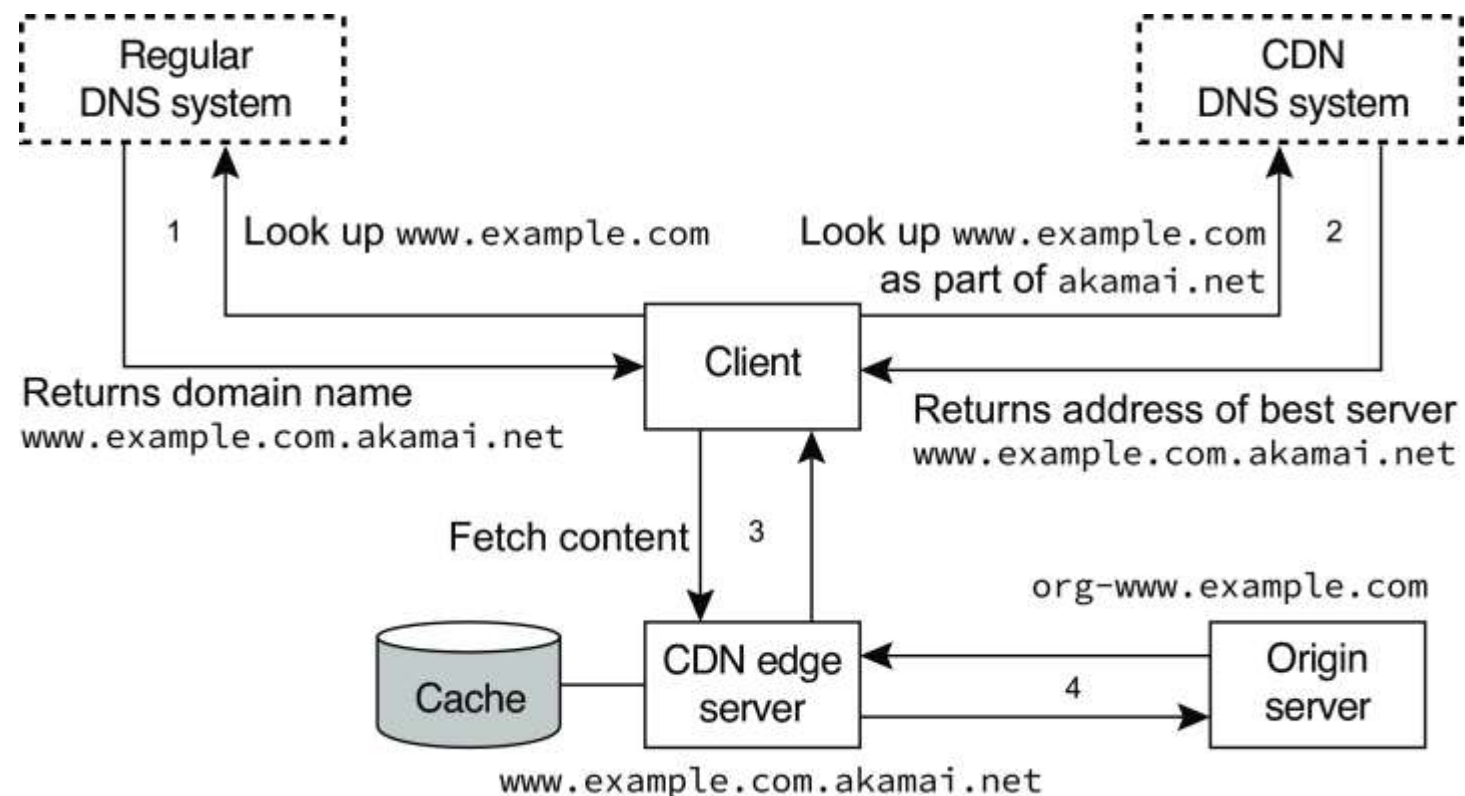


Important Note

The basic idea is that there is an origin server that hosts a Website with all its documents. To have the content of the origin server be hosted by a CDN, Akamai first makes sure that the domain name `www.example.com` is resolved to something like `www.example.com.akamai.net`, which, in turn, will refer to an edge server that is part of the Akamai CDN. To be able to still access the origin server, its domain name will have to be changed to, say, `org-www.example.com`.

Server Clusters

A Simplified Version of the Akamai CDN (3/5)

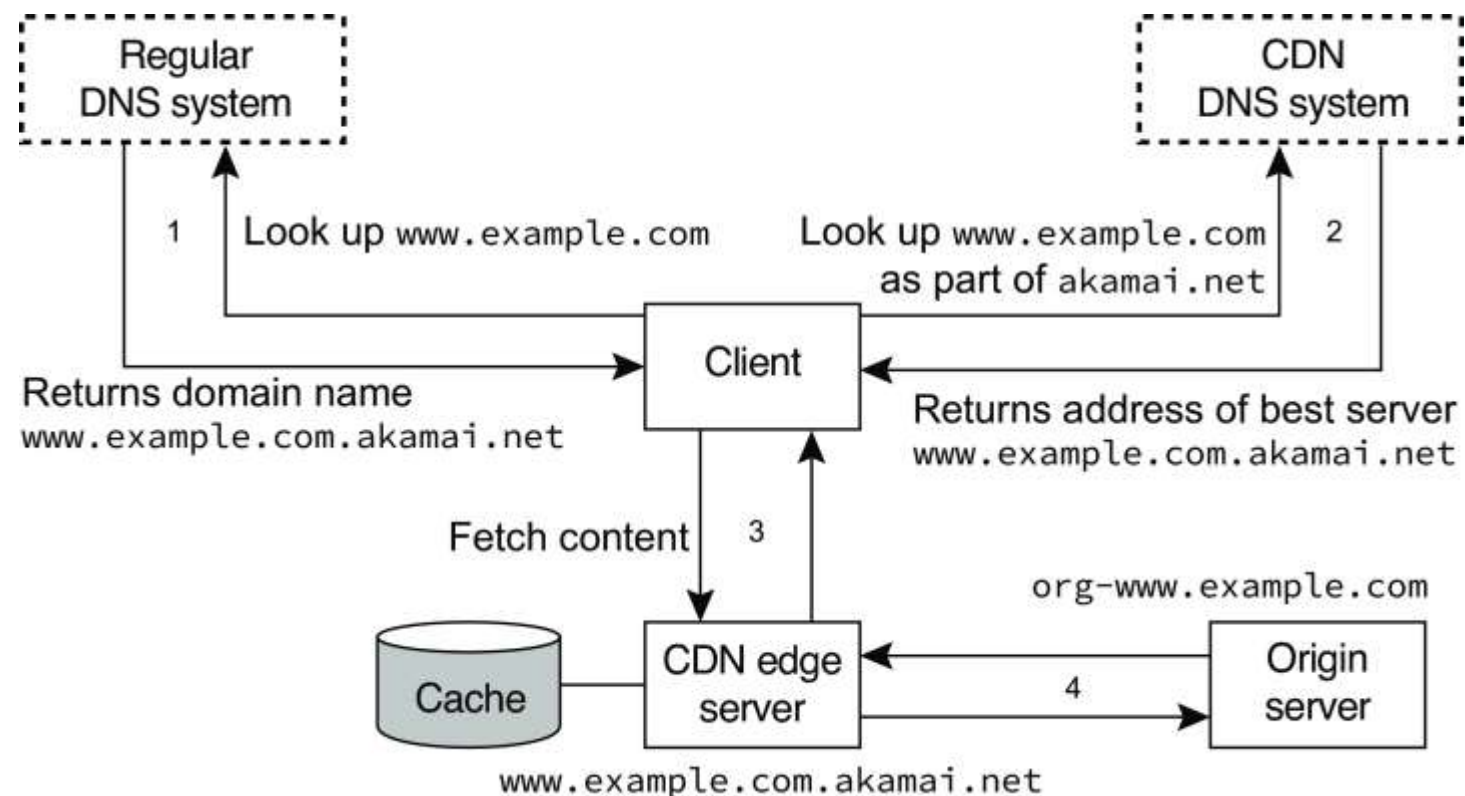


Important Note

The client first looks up the regular domain name, but is redirected to the akamai.net resolvers (step 1). The Akamai name resolvers will look up the best edge server to serve the client, and return its network address (step 2). This allows the client to contact the edge server (step 3), who is aware of the new name of the origin server. If the requested content is not in the edge server's cache, it fetches documents from the origin server (step 4), caches those documents and returns the requested ones to the client.

Server Clusters

A Simplified Version of the Akamai CDN (4/5)

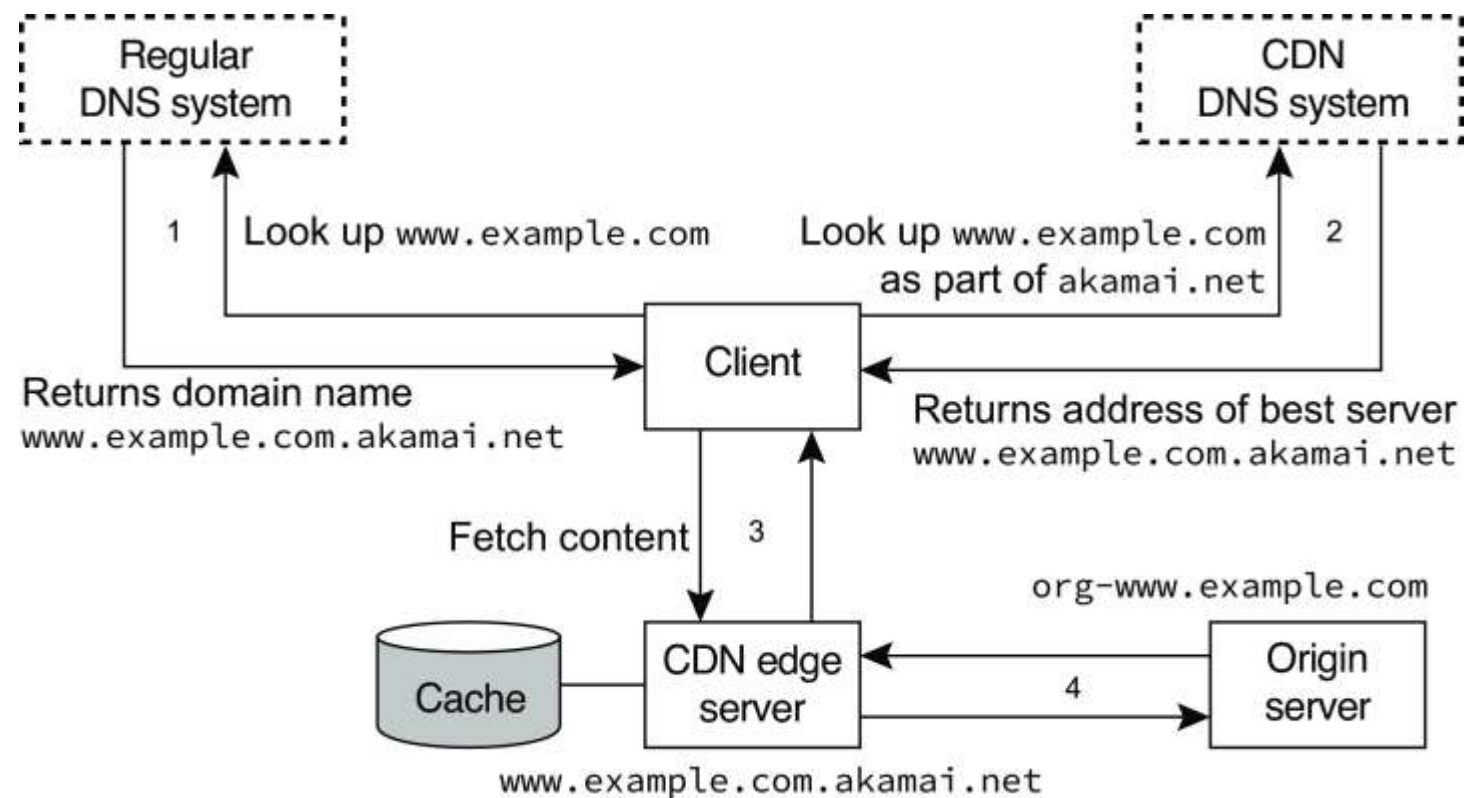


Important Note

In principle, by properly redirecting clients, a CDN can stay in control when it comes to client-perceived performance, but also considering global system performance by, for example, avoiding that requests are sent to heavily loaded servers. These so-called adaptive redirection policies can be applied when information on the system's current behavior is provided to the processes that take redirection decisions.

Server Clusters

A Simplified Version of the Akamai CDN (5/5)



Important Note

The cache in Akamai is often sophisticated enough to hold more than just passive data. Much of the application code of the origin server can be moved to the cache as well.

Code Migration

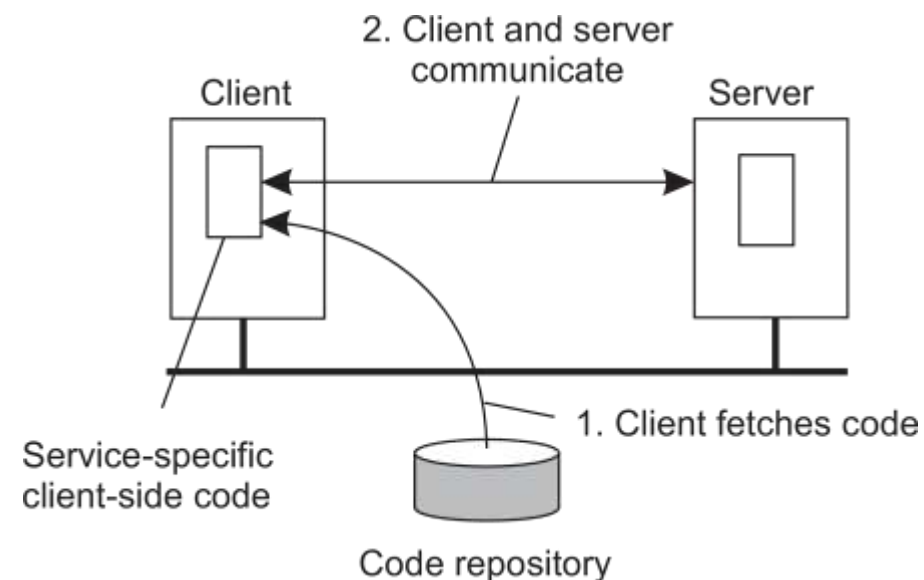
Code Migration

Reasons to Migrate Code (1/2)

Load Distribution

- The basic idea is that overall system performance can be improved if processes are moved from heavily loaded to lightly loaded machines.
- Ensuring that servers in a data center are sufficiently loaded (e.g., to prevent waste of energy)
- Minimizing communication by ensuring that computations are close to where the data is (think of mobile computing).

Flexibility: moving code to a client when needed



Avoids pre-installing software and increases dynamic configuration.

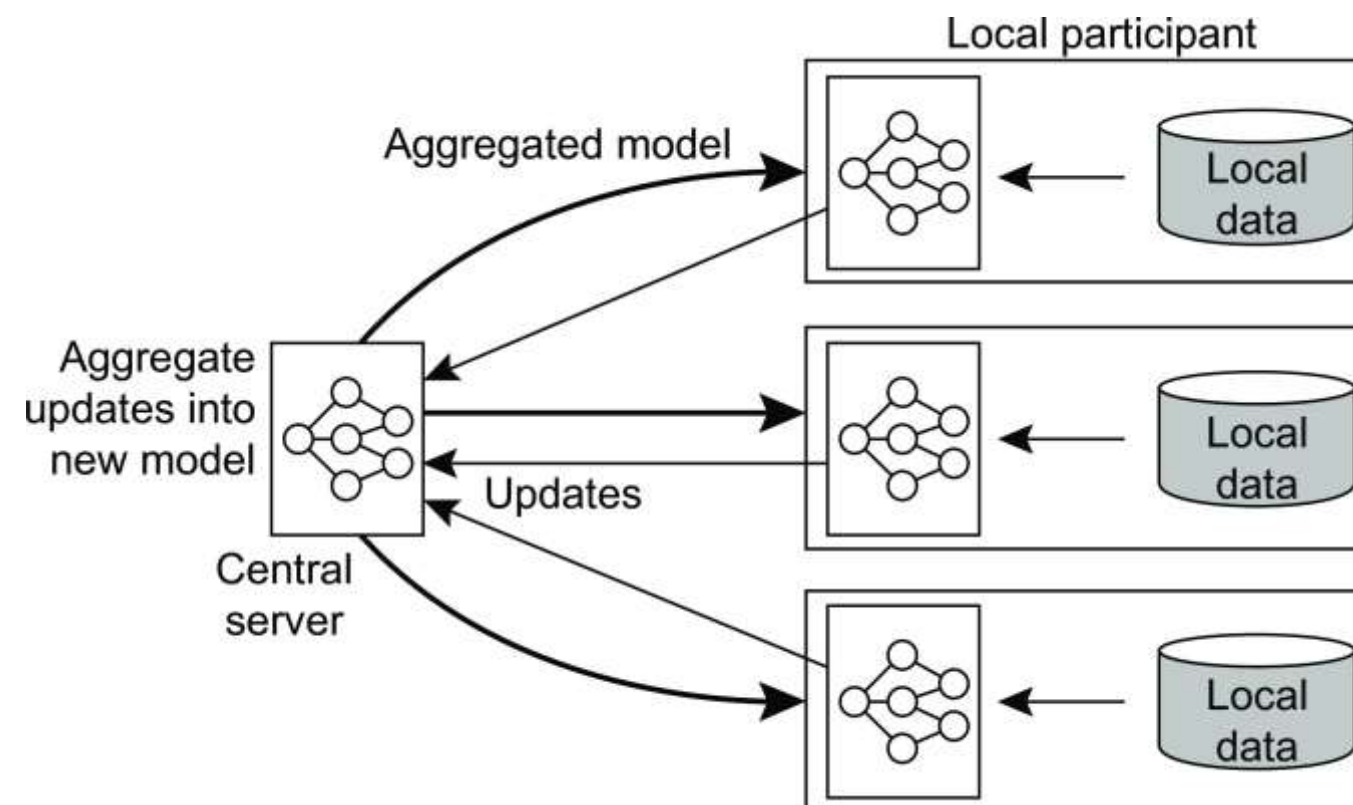
Code Migration

Reasons to Migrate Code (2/2)

Privacy and Security

In many cases, one cannot move data to another location, for whatever reason (often legal ones).
Solution: move the code to the data.

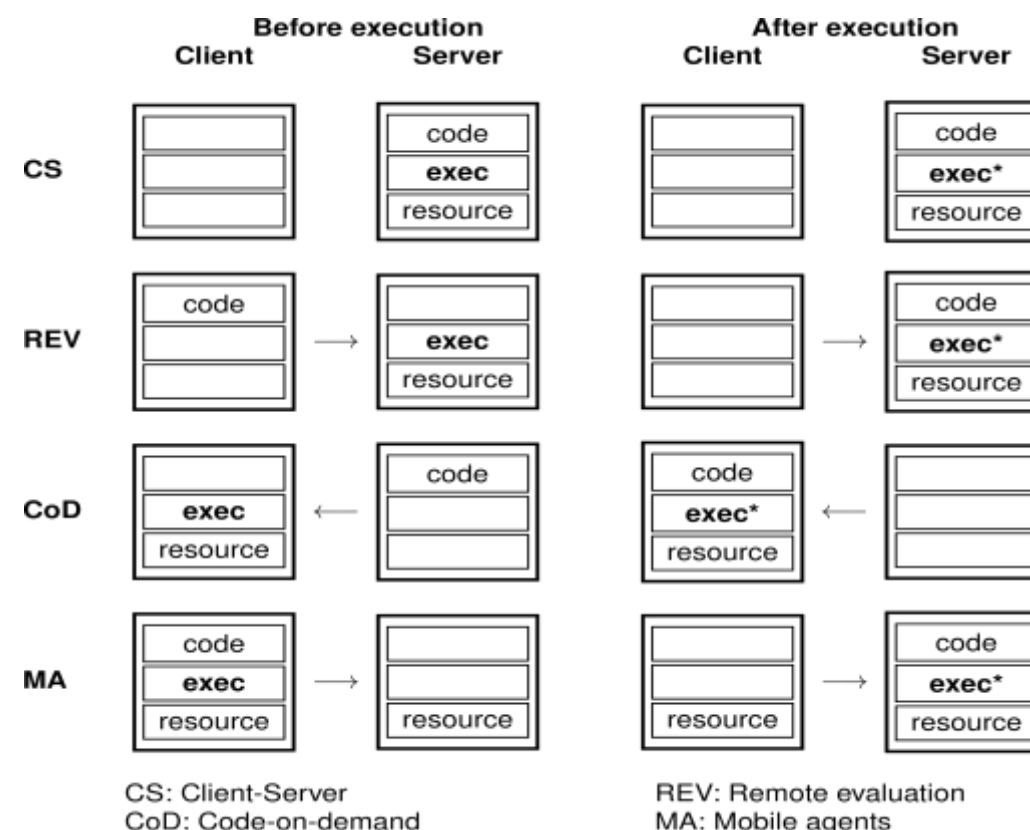
Example: Federated machine learning



Code Migration

Models

In the case of client-server computing, the code, execution state, and resource segment are all located at the server, and after execution, only the execution state at the server is generally modified. This state modification is denoted using an asterisk. With the sender-initiated remote evaluation, the client migrates code to the server where that code is executed and leading to a modification of the execution state at the server. Code-on-demand is a receiver-initiated scheme by which the client obtains code from the server, with its execution modifying the client-side execution state and operating on the client's resources. Finally, mobile agents typically follow a sender-initiated approach, moving code as well as execution state from the client to the server, operating on both the client's and the server's resources. Running a mobile agent will generally lead to modification of the associated execution state.



Code Migration

Models: Strong and weak mobility

Object components

- Code segment: contains the actual code
- Resource segment: contains references to external resources needed by the process, such as files, printers, devices, other processes
- Execution state: contains context of thread executing the object's code

Weak mobility: Move only code and resource segment (and reboot execution)

- Relatively simple, especially if code is portable
- Distinguish code shipping (push) from code fetching (pull)

Strong mobility: Move component, including execution state

- Migration: move entire object from one machine to the other
- Cloning: start a clone, and set it in the same execution state.

Code Migration in Heterogenous Systems

Main Problem

- The target machine may not be suitable to execute the migrated code
- The definition of process/thread/processor context is highly dependent on local hardware, operating system and runtime system
- Distributed systems are constructed on a heterogeneous collection of platforms, each having their own operating system and machine architecture.

Solution: abstract machine implemented on different platforms

- Interpreted languages, effectively having their own VM
- Virtual machine monitors
- Rely on a (process) virtual machine that either directly interprets source code (as in the case of scripting languages), or otherwise interprets intermediate code generated by a compiler (as in Java).
- Migrate not only processes, but to migrate entire computing environments. With virtual machine migration, it becomes possible to decouple a computing environment from the underlying system and actually migrate it to another machine.

Code Migration in Heterogenous Systems

Migrating a Virtual Machine

Migrating Images: three ways to handle migration

1. Pushing memory pages to the new machine and resending the ones that are later modified during the migration process.
2. Stopping the current virtual machine; migrate memory, and start the new virtual machine.
3. Letting the new virtual machine pull in new pages as needed: processes start on the new virtual machine immediately and copy memory pages on demand.

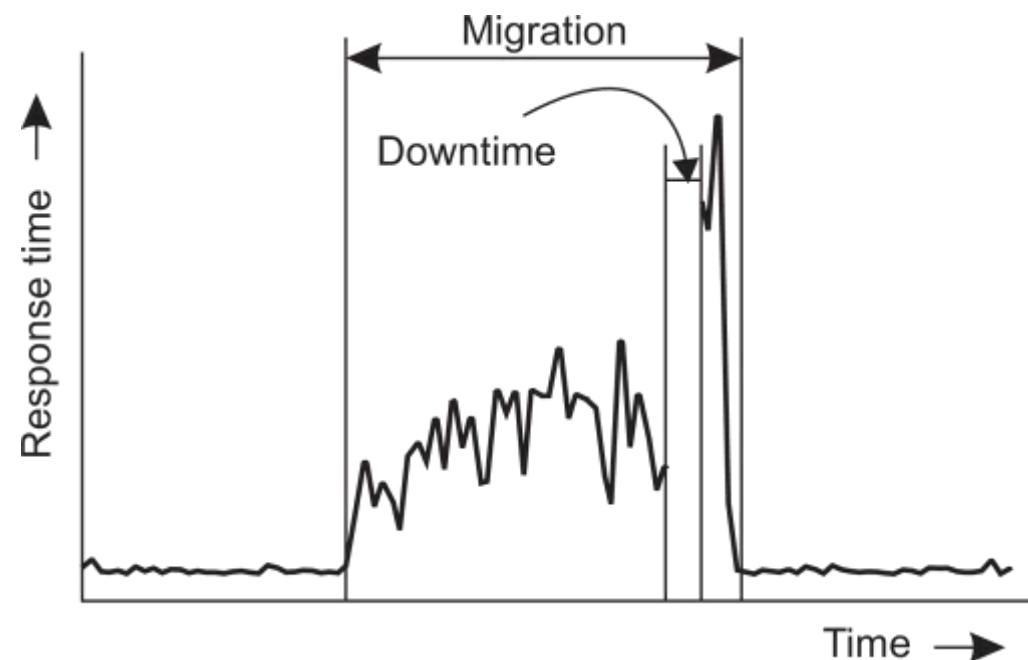
Code Migration in Heterogenous Systems

Performance of Migrating Virtual Machines

Problem

A complete migration may actually take tens of seconds. We also need to realize that during the migration, a service will be completely unavailable for multiple seconds. The good news is that the response time goes up significantly only after the downtime to complete the migration

Measurements regarding response times during VM migration



Thank You



MOVE FORWARD.
BE GREAT.